

Guia Completo: DDD, NestJS e Microserviços

Do Domínio ao Deploy

Fundamentos Estratégicos, Aplicação Prática e Arquitetura Distribuída

DeepStudy PDF Generator

August 28, 2026

Contents

Nota de Abertura	3
1 Parte 1 — Fundamentos do Domain-Driven Design (DDD)	4
1.1 O que é DDD e por que ele existe	4
1.2 Entities vs Value Objects	6
1.3 Aggregate vs Aggregate Root	9
1.4 Repositories: o contrato entre domínio e persistência	13
1.5 Domain Services vs Application Services	16
1.6 Domain Events	18
1.7 Bounded Contexts e Context Mapping	20
1.8 Camadas de uma aplicação DDD	22
2 Parte 2 — Conectando DDD ao Código: NestJS	24
2.1 Módulos, Controllers e Providers: a espinha dorsal do Nest	24
2.2 Injeção de Dependência e Escopos de Provider	27
2.3 Pipes, Guards, Interceptors e Exception Filters: o pipeline de uma requisição	29
2.4 DTOs: a fronteira validada	32
2.5 Decorators customizados e metadata com Reflector	35
2.6 Mappers e Anticorruption Layer: a tradução entre DTO e domínio	37
2.7 Aplicação Prática: estruturando DDD dentro de um projeto NestJS	41
2.8 CQRS com @nestjs/cqrs	44
2.9 Testes no NestJS	46
3 Parte 3 — Do Monólito aos Microsserviços	50

3.1	O que são microsserviços e quando NÃO usar	50
3.2	Comunicação síncrona vs assíncrona	52
3.3	NestJS Microservices: @MessagePattern vs @EventPattern	53
3.4	Consistência eventual: o padrão Saga	55
3.5	Quando usar eventos assíncronos — e quando NÃO usar	58
3.6	API Gateway, BFF e Circuit Breaker	58
3.7	Observabilidade distribuída	60
4	Conclusão	64
A	Referências e Leitura Adicional	66
B	Glossário	68
	Index	71

Nota de Abertura

Dica de Pleno

Este material assume que você já sabe escrever uma API em Node.js/TypeScript que funciona. O objetivo aqui é outro: aprender a **modelar o domínio antes de modelar o banco** (Parte 1), estruturar esse modelo dentro de um framework opinativo (Parte 2 — NestJS), e, quando fizer sentido, distribuir esse sistema em múltiplos serviços (Parte 3) — entendendo em cada passo *por que* essas decisões existem e *quando* elas custam mais caro do que valem. O guia responde, na ordem em que normalmente aparecem na cabeça de quem está aprendendo: o que é a fronteira de consistência do meu domínio (Aggregate)? como um pedaço do sistema avisa o resto que algo aconteceu, sem acoplar tudo (Domain Events e Saga)? como dado bruto de fora vira um objeto de domínio confiável (DTO e Mapper)? onde, fisicamente, cada peça mora dentro de um projeto NestJS real? e, por fim, essa estrutura deveria virar um único processo ou vários serviços independentes?

Chapter 1

Parte 1 — Fundamentos do Domain-Driven Design (DDD)

1.1 O que é DDD e por que ele existe

Domain-Driven Design, termo cunhado por Eric Evans no livro *Domain-Driven Design: Tackling Complexity in the Heart of Software* (2003), não é um framework, uma biblioteca ou um conjunto de pastas — é uma **abordagem de modelagem** que parte de uma premissa simples: a complexidade real de um sistema de software não está na tecnologia (banco, fila, cache), está nas **regras de negócio**. Um sistema de e-commerce não é difícil por causa do Express ou do Postgres; é difícil porque “cancelar um pedido” tem 7 regras diferentes dependendo do status do pagamento, da transportadora e de políticas de reembolso.

DDD ataca essa complexidade com duas ferramentas centrais:

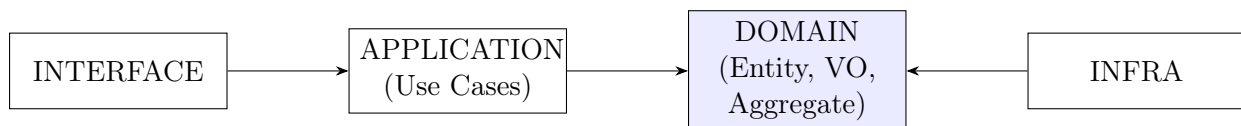
- **Ubiquitous Language (Linguagem Ubíqua):** o time de negócio e o time técnico devem usar exatamente os mesmos termos, e esses termos devem aparecer **no código**, não só em documentos. Se o time de negócio fala “pedido é *confirmado*, depois *faturado*, depois *despachado*”, o código deve ter uma classe `Pedido` com métodos `confirmar()`, `faturar()`, `despachar()` — não um `status: string` genérico com valores mágicos `'2'`, `'3'`, `'4'` que só um analista de negócio decodifica de cabeça.
- **Bounded Context (Contexto Delimitado):** a mesma palavra pode significar coisas diferentes em partes diferentes do sistema, e isso é **esperado**, não um erro de modelagem. “Cliente” no contexto de Vendas tem CPF, endereço de entrega e histórico de compras; “Cliente” no contexto de Cobrança tem dados bancários e score de crédito. Tentar criar uma única entidade `Cliente` “universal” que sirva aos dois contextos é o erro mais comum de quem começa em DDD — isso gera uma classe gigante, acoplada, que todo mundo tem medo de mexer. A seção 1.7 aprofunda como esses contextos se comunicam entre si.

Dica de Pleno

A pergunta que separa quem “leu sobre DDD” de quem “aplica DDD” é: você consegue apontar, no seu sistema atual, onde a linguagem do código diverge da linguagem que o time de negócio usa em uma reunião? Se o PO fala “assinatura pausada” e o código tem `subscription.active = false`, você já perdeu parte do valor do DDD — o código não comunica mais a regra, só um booleano genérico que exige ler a lógica ao redor para saber o que significa.

Mapa rápido: onde cada conceito desta parte se encaixa

As seções 1.2 a 1.7 introduzem peças (Entity, Value Object, Aggregate, Repository, Domain Service, Domain Event, Bounded Context) que, juntas, se organizam nas 4 camadas abaixo — a versão completa desse diagrama, com mais detalhe, está na seção 1.8, mas vale ter esse mapa em mente desde já:



DOMAIN = coração do DDD: 100% livre de framework, banco ou HTTP

Por Baixo dos Panos

DDD nasceu como reação a um padrão comum nos anos 2000: modelagem “anêmica” guiada pelo banco de dados. O time desenhava as tabelas primeiro (normalizadas, sem duplicação, otimizadas para armazenamento) e depois gerava classes que eram só espelhos dessas tabelas — objetos com getters/setters e nenhuma regra de negócio, também chamados de **Anemic Domain Model** (um “anti-padrão” nomeado pelo próprio Martin Fowler). O problema: as regras de negócio não desaparecem, elas só migram para outro lugar — geralmente para dentro de “Services” gigantes cheios de ifs, ou pior, espalhadas em controllers. DDD propõe o oposto: modele primeiro o **comportamento** do domínio (o que um Pedido *pode fazer*, não só quais campos ele tem), e deixe o esquema de banco ser uma consequência, não o ponto de partida.

Cenário de Falha em Produção

Um sistema de assinaturas (SaaS B2B) modelado de forma anêmica: uma classe `Assinatura` com só propriedades públicas (`status`, `dataFim`, `plano`) e toda a lógica de “posso fazer upgrade?”, “o que acontece se o pagamento falhar 3 vezes?”, “posso reativar depois de cancelada?” espalhada em 4 services diferentes (`AssinaturaService`, `PagamentoService`, `NotificacaoService`, `RelatorioService`), cada um checando `if (assinatura.status === 'cancelada' && assinatura.dataFim > hoje)`

com sua própria variação da mesma regra. Um dia, o time decide que assinatura cancelada só pode ser reativada em até 7 dias — mas como a regra estava duplicada em 4 lugares com pequenas variações, o `PagamentoService` foi atualizado, mas o `RelatorioService` continuou contando reativações fora da janela de 7 dias como válidas, inflando métricas de retenção reportadas para o board por 2 meses até alguém notar a divergência entre o painel financeiro e o operacional.

Dica de Produção

Um sinal de alerta em produção (não no código, no **comportamento observável** do sistema) de que o modelo está anêmico: bugs de “regra de negócio aplicada em um lugar mas não em outro” — dois endpoints que deveriam impor a mesma regra, mas um deles esqueceu. Se isso acontece com frequência no seu sistema, é sintoma de que a regra deveria estar **dentro** da entidade (um método `assinatura.podeReativar()` chamado de todo lugar), não replicada em cada service que toca aquele objeto. Em code review, trate “esse `if` de regra de negócio já existe em outro arquivo?” como uma pergunta obrigatória, não opcional.

1.2 Entities vs Value Objects

Todo o material que segue — Aggregates, Domain Events, DTOs, Mappers — depende de uma distinção que precisa estar automática antes de avançar: DDD separa objetos de domínio em dois tipos fundamentais, e tratá-los como intercambiáveis é o erro de modelagem mais comum e mais caro de corrigir depois.

- **Entity (Entidade):** tem **identidade** própria, que persiste ao longo do tempo mesmo que seus atributos mudem. Dois `Pedidos` com exatamente os mesmos itens e o mesmo valor ainda são pedidos **diferentes** se têm IDs diferentes — e o mesmo `Pedido`, depois de ter um item alterado, continua sendo “o mesmo pedido”. **Identidade, não atributos, define igualdade.**
- **Value Object (Objeto de Valor):** **não tem identidade**, é definido inteiramente pelos seus atributos, e deve ser **imutável**. Dois objetos `Dinheiro(100, 'BRL')` são **iguais** entre si — não importa “qual instância” é qual, importa o valor. Se você precisa “mudar” um Value Object, você cria um novo, nunca muta o existente.

```
// npx ts-node entity-vs-value-object.ts

// === VALUE OBJECT: imutável, igualdade por valor, sem identidade própria ===
class Dinheiro {
  private readonly centavos: number;
  private readonly moeda: string;

  constructor(centavos: number, moeda: string) {
    if (centavos < 0) throw new Error('Dinheiro não pode ser negativo');
  }
}
```

```
this.centavos = centavos;
this.moeda = moeda;
}

somar(outro: Dinheiro): Dinheiro {
  if (outro.moeda !== this.moeda) throw new Error('Moedas incompatíveis');
  return new Dinheiro(this.centavos + outro.centavos, this.moeda); // NOVA
  ↪ instância
}

igual(outro: Dinheiro): boolean {
  return this.centavos === outro.centavos && this.moeda === outro.moeda; //
  ↪ por VALOR
}

formatar(): string {
  return `${this.moeda} ${(this.centavos / 100).toFixed(2)}`;
}
}

// === ENTITY: identidade própria (id), igualdade por identidade, é mutável ===
class ItemPedido {
  constructor(
    public readonly id: string,
    private nome: string,
    private preco: Dinheiro,
  ) {}

  igual(outro: ItemPedido): boolean {
    return this.id === outro.id; // por IDENTIDADE, não por nome/preço
  }

  precoAtual(): Dinheiro {
    return this.preco;
  }
}

const preco1 = new Dinheiro(5000, 'BRL');
const preco2 = new Dinheiro(5000, 'BRL');
console.log('Dois VOs com mesmo valor são iguais?', preco1.igual(preco2));

const item1 = new ItemPedido('item-1', 'Teclado', preco1);
const item2 = new ItemPedido('item-1', 'Teclado (nome mudou)', new
  ↪ Dinheiro(6000, 'BRL'));
console.log('Duas Entities com o MESMO id são iguais?', item1.igual(item2));
```

Retorno esperado

```
Dois VOs com mesmo valor são iguais? true
Duas Entities com o MESMO id são iguais? true
```

Dica de Pleno

A segunda linha do retorno é intencionalmente contraintuitiva: `item1` e `item2` têm nome e preço diferentes, mas ainda são o “mesmo item” porque compartilham `id`. Isso é o ponto central de uma Entity — ela representa algo que **muda ao longo do tempo continuando sendo a mesma coisa** (como você, com cabelo diferente do de 10 anos atrás, mas ainda “você” pelo seu CPF). Um Value Object como `Dinheiro` não tem esse conceito: `Dinheiro(100, 'BRL')` de ontem e de hoje são **indistinguíveis**, e é isso que permite compará-los por valor com segurança.

Por que a imutabilidade do Value Object não é só estilo

A imutabilidade de um VO não é uma preferência de código limpo — é o que garante que, uma vez validado no construtor, um `Dinheiro` **permanece válido para sempre**, sem chance de alguém mutar `centavos` para um valor negativo em algum ponto distante do código. Esse princípio tem nome consolidado: “**Parse, don’t validate**” — valide uma vez, na borda de construção, e deixe o próprio tipo carregar essa garantia adiante, em vez de revalidar em todo lugar que usa o dado. Esse mesmo princípio reaparece, sob outro nome, quando chegarmos a DTOs (seção 2.4) — é a mesma disciplina, aplicada em outra fronteira.

Armadilha Comum

Modelar CPF, e-mail, dinheiro, endereço ou intervalos de data como `string/number` soltos é o erro mais comum de quem ainda não internalizou Value Objects. O problema não é só estético: um `precoEmCentavos: number` aceita `-500`, `NaN`, ou ser somado com uma `quantidade: number` sem o compilador reclamar (ambos são `number`). Um Value Object fecha essa porta na fronteira de construção — depois de criado, ele é garantido válido para sempre.

Cenário de Falha em Produção

Um sistema de logística representa “período de entrega” como dois campos soltos, `dataInicio: Date` e `dataFim: Date`, espalhados em 6 classes diferentes. Cada lugar que usa esses campos reimplementa sua própria checagem de “o período é válido?” (`dataFim > dataInicio`). Um novo fluxo de reagendamento monta esses dois campos a partir de input do usuário e esquece a checagem — pedidos com `dataFim` anterior a `dataInicio` entram no banco, e um job noturno que calcula “dias em atraso” começa a gerar números negativos que quebram o dashboard operacional, sem exceção nenhuma lançada em lugar algum.

Dica de Produção

Se um bug de produção tem como causa raiz “esse dado inválido não deveria nunca ter chegado até aqui”, é quase sempre sinal de que faltou um Value Object na fronteira certa. Não corrija só o sintoma (um `if` isolado no lugar que quebrou) — pergunte “que outros lugares constroem esse mesmo par de campos, e será que

todos validam igual?”.

1.3 Aggregate vs Aggregate Root

Duas palavras, dois conceitos — não confunda

Esta é a distinção mais mal compreendida do DDD tático, e vale isolar antes de qualquer exemplo de código:

Termo	O que é
Aggregate	O cluster inteiro de objetos (Entities + Value Objects) tratado como uma unidade de consistência. No exemplo abaixo, o Aggregate é o conjunto Pedido + todos os seus ItemPedido .
Aggregate Root	A única Entity dentro do cluster que pode ser referenciada de fora, e o único ponto de entrada para qualquer leitura ou modificação nos objetos internos ao Aggregate. No exemplo abaixo, a Aggregate Root é especificamente o Pedido — não o cluster inteiro, só a entidade-porta.

Em outras palavras: **Aggregate é o território; Aggregate Root é o único portão de entrada desse território**. Ninguém de fora entra por uma janela lateral (acessando um **ItemPedido** diretamente) — todo mundo entra pela porta da frente (**Pedido**), e é a porta quem decide se a entrada é permitida, validando as invariantes do território inteiro.

Dica de Pleno

Um erro comum de quem está aprendendo é dizer “o **Pedido** é o **Aggregate**” — tecnicamente impreciso. O **Pedido** é a **Aggregate Root**; o **Aggregate** é **Pedido** + a coleção de **ItemPedidos** + qualquer outro Value Object interno ao cluster. Todo Aggregate Root é uma Entity, mas nem toda Entity é uma Aggregate Root — só é raiz a Entity que serve como porta de entrada do cluster.

A regra de ouro: tudo passa pela raiz

A regra mais importante, e a que mais gera bugs quando violada: **objetos internos ao Aggregate nunca são acessados ou modificados diretamente de fora — sempre através da Aggregate Root**. Ninguém de fora chama `itemPedido.alterarQuantidade(5)` diretamente; quem chama é `pedido.alterarQuantidadeDoItem(itemId, 5)`, porque só o **Pedido** (a raiz) sabe garantir invariantes que dependem do cluster inteiro, como “a soma dos itens não pode ultrapassar um limite”.

```
// ENTITY INTERNA ao Aggregate - nunca é acessada diretamente de fora
class ItemPedido {
  constructor(
    public readonly produtoId: string,
    private quantidade: number,
    private precoUnitarioCentavos: number,
  ) {}

  get subtotal(): number {
    return this.quantidade * this.precoUnitarioCentavos;
  }

  alterarQuantidade(novaQuantidade: number): void {
    if (novaQuantidade <= 0) throw new Error('Quantidade deve ser positiva');
    this.quantidade = novaQuantidade;
  }
}

// PEDIDO é a AGGREGATE ROOT - a ÚNICA porta de entrada do Aggregate inteiro
class Pedido {
  private itens: ItemPedido[] = []; // parte do Aggregate, mas NUNCA exposto
  ↪ diretamente
  private status: 'ABERTO' | 'CONFIRMADO' = 'ABERTO';

  constructor(public readonly id: string, private readonly limiteItensCentavos:
    ↪ = 500_000) {}

  adicionarItem(produtoId: string, quantidade: number, precoUnitarioCentavos:
    ↪ number): void {
    if (this.status !== 'ABERTO') {
      throw new Error('Não é possível adicionar itens a um pedido já
        ↪ confirmado'); // invariante
    }
    const item = new ItemPedido(produtoId, quantidade, precoUnitarioCentavos);
    const novoTotal = this.calcularTotal() + item.subtotal;
    if (novoTotal > this.limiteItensCentavos) {
      throw new Error('Total do pedido ultrapassaria o limite permitido'); //
        ↪ invariante
    }
    this.itens.push(item);
  }

  // MODIFICAR um item SEMPRE passa pela raiz - nunca se expõe o array de itens
  alterarQuantidadeDoItem(produtoId: string, novaQuantidade: number): void {
    const item = this.itens.find((i) => i.produtoId === produtoId);
    if (!item) throw new Error('Item não encontrado neste pedido');
    item.alterarQuantidade(novaQuantidade);
  }

  confirmar(): void {
```

```
    if (this.itens.length === 0) throw new Error('Pedido sem itens não pode ser  
    ↪ confirmado');  
    this.status = 'CONFIRMADO';  
  }  
  
  calcularTotal(): number {  
    return this.itens.reduce((soma, item) => soma + item.subtotal, 0);  
  }  
}  
  
const pedido = new Pedido('pedido-1');  
pedido.adicionarItem('produto-a', 2, 10_000);  
pedido.adicionarItem('produto-b', 1, 30_000);  
console.log('Total após adicionar itens:', pedido.calcularTotal());  
  
pedido.confirmar();  
try {  
  pedido.adicionarItem('produto-c', 1, 5_000); // deve falhar: pedido já  
  ↪ confirmado  
} catch (erro) {  
  console.log('Erro esperado:', (erro as Error).message);  
}
```

Retorno esperado

```
Total após adicionar itens: 50000  
Erro esperado: Não é possível adicionar itens a um pedido já confirmado
```

Dica de Pleno

Repare que não existe um método `pedido.getItems()` devolvendo o array mutável de `ItemPedido`. Se existisse, qualquer código externo poderia fazer `pedido.getItems()[0].alterarQuantidade(999)` diretamente, **contornando** a checagem de `limiteItensCentavos` — uma “porta lateral” que anularia toda a proteção da Aggregate Root. É por isso que a raiz é chamada de raiz: ela é o único caminho de acesso permitido para tudo que está dentro do território do Aggregate.

Projete Aggregates pequenos

O tamanho do Aggregate é a decisão de modelagem mais debatida na comunidade DDD, e a diretriz mais aceita (defendida por Vaughn Vernon em *Implementing Domain-Driven Design*) é “**projete Aggregates pequenos**”. A tentação inicial é modelar um Aggregate `Pedido` gigante que inclui `Cliente`, `Pagamento` e `Entrega` inteiros dentro dele — mas o tamanho do Aggregate é, na prática, o que fica travado (via lock otimista) durante uma transação de escrita. Um Aggregate grande significa mais contenção: dois usuários alterando partes “independentes” (endereço de entrega vs status de pagamento) do mesmo Aggregate gigante colidem em concorrência mesmo sem conflito de negócio real. A prática

recomendada é referenciar **outros** Aggregates por ID (`pedido.clienteId: string`), nunca por composição direta (`pedido.cliente: Cliente`).

Por Baixo dos Panos

Um Aggregate mal dimensionado se manifesta tecnicamente como erros de **concorrência otimista** (`version mismatch`, `stale object`) em bancos que versionam registros — porque toda escrita em qualquer parte do cluster incrementa a versão do registro raiz inteiro. Se dois `ItemPedidos` de um mesmo `Pedido` são alterados por dois usuários ao mesmo tempo (cenário raro, mas plausível em pedidos colaborativos), a segunda escrita colide com a primeira mesmo que os itens alterados sejam diferentes — porque, do ponto de vista do banco, é o mesmo registro-raiz sendo versionado.

Cenário de Falha em Produção

Um sistema de reservas de hotel modela `Hotel` como Aggregate Root contendo a lista completa de `Quartos` e `Reservas` do hotel inteiro dentro do mesmo Aggregate — “logicamente pertencem ao hotel”. Em alta temporada, duas requisições concorrentes reservando quartos **diferentes** do mesmo hotel colidem em erro de concorrência otimista, mesmo não havendo conflito de negócio nenhum entre elas — porque ambas tentam salvar uma nova versão do mesmo Aggregate `Hotel` gigante. O resultado observável é uma taxa de erro 5xx elevada exatamente nos hotéis mais populares, no horário de pico de reservas.

Dica de Produção

Se seus logs de erro mostram picos de conflito de concorrência otimista concentrados em registros específicos de alto tráfego, a pergunta certa não é “como aumento o timeout do lock” — é “esse Aggregate está modelado grande demais, misturando coisas que só *parecem* relacionadas mas mudam de forma independente?”. No caso do hotel, `Quarto` deveria ser seu próprio Aggregate (com sua própria raiz), e `Reserva` deveria referenciar `quartoId` por ID, não estar aninhada dentro do Aggregate `Hotel`.

Desafio de Mentoria

Você está modelando um sistema de biblioteca. Um `Livro` tem várias `Cópias` físicas, e cada `Cópia` pode estar emprestada a um `Usuário`. Um colega propõe: “`Livro` é a Aggregate Root, e `Cópia` fica dentro dele — assim garantimos, dentro do Aggregate, que não emprestamos mais cópias do que existem”. Como Advogado do Diabo: em uma biblioteca com 50 mil títulos e milhares de empréstimos simultâneos por dia, que problema de concorrência essa modelagem específica provavelmente vai gerar? Você modelaria `Cópia` como Aggregate próprio? Se sim, como garantir a invariante “não pode haver duas cópias com o mesmo código de barras” sem ela estar dentro do Aggregate `Livro`?

1.4 Repositories: o contrato entre domínio e persistência

Um **Repository** em DDD não é “a classe que fala com o banco” — é uma **abstração que simula uma coleção em memória** de Aggregate Roots. O domínio pede `repositorio.buscarPorId()` ou `repositorio.salvar(pedido)` sem saber, e sem se importar, se por trás disso existe Postgres, MongoDB ou um arquivo JSON. A interface do repositório é definida **dentro do domínio** (é uma “porta”); a implementação concreta (Prisma, TypeORM, SQL cru) vive na infraestrutura e **implementa** essa porta.

```
// npx ts-node repository-pattern.ts
```

```
class Pedido {
  constructor(public readonly id: string, private status: string) {}
  get statusAtual() { return this.status; }
}
```

```
// A INTERFACE vive no domínio - é o domínio quem dita o contrato, não o banco
interface PedidoRepository {
  buscarPorId(id: string): Promise<Pedido | null>;
  salvar(pedido: Pedido): Promise<void>;
}
```

```
// A IMPLEMENTAÇÃO vive na infraestrutura - sabe de "banco", o domínio não sabe
class PedidoRepositoryEmMemoria implements PedidoRepository {
  private dados = new Map<string, Pedido>();

  async buscarPorId(id: string): Promise<Pedido | null> {
    return this.dados.get(id) ?? null;
  }

  async salvar(pedido: Pedido): Promise<void> {
    this.dados.set(pedido.id, pedido);
  }
}
```

```
// CASO DE USO depende só da INTERFACE - nunca da implementação concreta
class ConfirmarPedidoUseCase {
  constructor(private repositorio: PedidoRepository) {}

  async executar(pedidoId: string): Promise<void> {
    const pedido = await this.repositorio.buscarPorId(pedidoId);
    if (!pedido) throw new Error('Pedido não encontrado');
    await this.repositorio.salvar(pedido);
  }
}
```

```
async function main() {
  const repo = new PedidoRepositoryEmMemoria();
```

```
await repo.salvar(new Pedido('p1', 'ABERTO'));

const useCase = new ConfirmarPedidoUseCase(repo);
await useCase.executar('p1');
console.log('Pedido processado com sucesso via repositório em memória');
}

main();
```

Retorno esperado

Pedido processado com sucesso via repositório em memória

Dica de Pleno

Essa inversão — a interface pertence ao domínio, a implementação pertence à infra — é uma aplicação direta do **Dependency Inversion Principle** (o “D” de SOLID). Na prática, isso significa que `ConfirmarPedidoUseCase` pode ser testado com um repositório em memória (como no exemplo), sem subir Postgres nenhum, e o mesmo caso de uso funciona em produção com um `PedidoRepositoryPrisma` real — a troca é só de qual implementação é injetada, zero mudança no caso de uso.

Armadilha Comum

Um erro comum é criar métodos de repositório “genéricos demais”, tipo `buscarPorFiltro(filtros: any)` que aceita qualquer combinação de campos — isso parece flexível, mas na prática vaza detalhes de query (nomes de coluna, operadores SQL) para dentro do domínio através do formato do filtro. Prefira métodos de repositório **nomeados pela intenção de negócio**: `buscarPedidosPendentesDoCliente(clienteId)`, não `buscar({ status: 'pendente', clienteId })`. O primeiro é auto-documentado e pode ser otimizado internamente (índice específico, query dedicada) sem quebrar contrato; o segundo empurra a responsabilidade de “montar a query certa” para quem chama.

Cenário de Falha em Produção

Um repositório com método genérico `buscar(where: Record<string, any>)` que repassa o objeto quase direto para a cláusula `WHERE` do ORM. Um endpoint de busca administrativa aceita filtros vindos de query string e os repassa, sem validação, para esse método — um usuário malicioso descobre que pode injetar um filtro `{ senha: { not: null } }` (sintaxe do Prisma) para inferir, campo por campo via respostas diferentes, dados que não deveriam ser filtráveis publicamente, um tipo de vazamento por **over-fetching via filtro não controlado**, prima do SQL Injection mas na camada de ORM.

Dica de Produção

Repositórios com métodos nomeados por intenção de negócio não são só “mais limpos” — são uma superfície de ataque menor por construção, porque cada método só aceita exatamente os parâmetros que o negócio precisa (um `clienteId: string`, não um objeto de filtro arbitrário). Em auditoria de segurança, trate qualquer método de repositório que aceite um objeto de filtro genérico vindo (direta ou indiretamente) de input do usuário como prioridade de revisão — é onde SQL/NoSQL Injection e over-fetching mais aparecem em sistemas com ORM.

Transações e o padrão Unit of Work

Um ponto prático que o exemplo acima simplifica: quando confirmar um `Pedido` também precisa, na mesma transação de banco, debitar estoque e registrar uma entrada financeira (ou seja, quando a operação toca **mais de um repositório**), quem abre e fecha essa transação? A resposta do DDD é o padrão **Unit of Work** — um objeto que rastreia as mudanças feitas durante uma operação e as persiste como uma única transação atômica, garantindo que ou tudo é salvo, ou nada é. Em NestJS + Prisma, isso é, na prática, o `prisma.$transaction()`:

```
// application/confirmar-pedido.use-case.ts
@Injectable()
class ConfirmarPedidoUseCase {
  constructor(private readonly prisma: PrismaService) {}

  async executar(pedidoId: string): Promise<void> {
    await this.prisma.$transaction(async (tx) => {
      const pedidoRepo = new PedidoRepositoryPrisma(tx); // repositórios recebem
      ↪ o cliente TRANSACIONAL
      const estoqueRepo = new EstoqueRepositoryPrisma(tx);

      const pedido = await pedidoRepo.buscarPorId(pedidoId);
      pedido.confirmar();
      await pedidoRepo.salvar(pedido);
      await estoqueRepo.debitarItens(pedido.itens); // mesma transação - ou os
      ↪ dois, ou nenhum
    });
  }
}
```

Dica de Pleno

A responsabilidade de abrir a transação pertence ao **Application Service** (Use Case), nunca ao Domain Service nem à Entity — o domínio puro não deveria saber que “transação de banco” existe, isso é vocabulário de infraestrutura. É o Use Case quem sabe que essa operação específica precisa ser atômica entre dois repositórios,

porque é ele quem orquestra a chamada aos dois; a Entity `Pedido` continua não importando nada de Prisma, exatamente como reforça a regra de dependência da seção 1.8.

1.5 Domain Services vs Application Services

DDD distingue dois tipos de “service”, e a confusão entre eles é constante:

- **Domain Service:** contém uma regra de negócio que **não pertence naturalmente a nenhuma Entity ou Value Object específico**, geralmente porque envolve **mais de um agregado**. Exemplo: “transferir saldo entre duas contas” não pertence só à conta de origem nem só à de destino — é uma operação que orquestra as duas, então vira um Domain Service `ServicoDeTransferencia`. Continua sendo **domínio puro**: zero dependência de HTTP, banco, ou frameworks.
- **Application Service (ou “Use Case”):** orquestra a **infraestrutura** ao redor do domínio — busca agregados no repositório, chama o domínio (Entity, Value Object ou Domain Service), salva o resultado, dispara eventos, gerencia a transação. É a “cola” entre o mundo externo (um controller HTTP) e o domínio puro.

```
// npx ts-node domain-vs-application-service.ts
```

```
class Conta {
  constructor(public readonly id: string, private saldoCentavos: number) {}
  get saldo() { return this.saldoCentavos; }
  debitar(valor: number) {
    if (valor > this.saldoCentavos) throw new Error('Saldo insuficiente');
    this.saldoCentavos -= valor;
  }
  creditar(valor: number) { this.saldoCentavos += valor; }
}

// DOMAIN SERVICE - regra de negócio que envolve DUAS entidades, sem infra
↳ nenhuma
class ServicoDeTransferencia {
  transferir(origem: Conta, destino: Conta, valorCentavos: number): void {
    origem.debitar(valorCentavos); // regra pura: se saldo insuficiente, lança
    ↳ erro aqui
    destino.creditar(valorCentavos);
  }
}

interface ContaRepository {
  buscarPorId(id: string): Promise<Conta>;
  salvar(conta: Conta): Promise<void>;
}
```

```
// APPLICATION SERVICE (Use Case) - orquestra infra (repositório) + domínio
class TransferirEntreContasUseCase {
  constructor(
    private repositorio: ContaRepository,
    private domainService: ServicoDeTransferencia,
  ) {}

  async executar(origemId: string, destinoId: string, valorCentavos: number):
    ↪ Promise<void> {
    const origem = await this.repositorio.buscarPorId(origemId);
    const destino = await this.repositorio.buscarPorId(destinoId);

    this.domainService.transferir(origem, destino, valorCentavos); // delega a
    ↪ REGRA para o domínio

    await this.repositorio.salvar(origem);
    await this.repositorio.salvar(destino);
  }
}
```

Dica de Pleno

O teste decisivo para saber se algo é Domain Service ou Application Service: um Domain Service, se você tirar o `async/await` e qualquer chamada a repositório, **ainda faz sentido sozinho** — é lógica pura, testável sem mock nenhum. Se o método faz `await repositorio.algo()`, ele **não** é Domain Service, é Application Service (orquestração). Ver `await` dentro de uma classe que deveria ser “domínio puro” é o sinal mais confiável de que a responsabilidade está no lugar errado.

Cenário de Falha em Produção

Um `TransferirEntreContasUseCase` que, além de orquestrar, também **contém a regra de negócio inline** (`if (origem.saldo < valor) throw ...` direto no use case, em vez de delegar para o domínio) — funciona bem até que uma segunda forma de transferência aparece no sistema (transferência agendada, processada por um worker/cron, não por um controller HTTP). Como a regra de “saldo insuficiente” estava dentro do use case ligado ao fluxo HTTP, o novo worker de transferências agendadas reimplementa sua própria versão da regra — e diverge sutilmente (o worker esquece de checar limite diário de transferência, que o use case HTTP checava). Resultado: transferências agendadas conseguem ultrapassar o limite diário que transferências manuais respeitam, um bug de negócio silencioso descoberto só na auditoria financeira do trimestre.

Dica de Produção

Uma boa regra prática em revisão de código: se você está adicionando uma **segunda forma de entrada** para uma mesma regra de negócio (um novo endpoint, um worker, um job agendado) e percebe que vai “copiar e colar” lógica de um use case

existente, é sinal de que essa lógica deveria subir um nível — sair do Application Service e virar Domain Service (ou método da própria Entity), reutilizável por **qualquer** orquestrador, HTTP ou não.

1.6 Domain Events

Um **Domain Event** representa “algo importante que aconteceu no domínio”, modelado como um objeto imutável, nomeado **no passado**: `PedidoConfirmado`, `PagamentoRecusado`, `EstoqueEsgotado`. A ideia central: em vez de o código que confirma um pedido **também** saber que precisa enviar e-mail, atualizar métricas e notificar logística (acoplando o `Pedido` a serviços que não são da sua responsabilidade), a Aggregate Root só **emite o evento** — e quem quiser reagir se inscreve para ouvi-lo, sem o `Pedido` saber quem está ouvindo.

```
// npx ts-node domain-events.ts

// EVENTO: imutável, representa um fato que já aconteceu (nome no PASSADO)
class PedidoConfirmadoEvent {
  constructor(
    public readonly pedidoId: string,
    public readonly totalCentavos: number,
    public readonly ocorridoEm: Date = new Date(),
  ) {}
}

class Pedido {
  private eventos: PedidoConfirmadoEvent[] = [];

  constructor(public readonly id: string, private totalCentavos: number) {}

  confirmar(): void {
    // a Aggregate Root REGISTRA o evento - não sabe quem vai reagir a ele
    this.eventos.push(new PedidoConfirmadoEvent(this.id, this.totalCentavos));
  }

  liberarEventos(): PedidoConfirmadoEvent[] {
    const eventos = [...this.eventos];
    this.eventos = [];
    return eventos;
  }
}

// HANDLERS: cada um reage de forma independente, sem se conhecerem
function enviarEmailConfirmacao(evento: PedidoConfirmadoEvent) {
  console.log(`[E-MAIL] Pedido ${evento.pedidoId} confirmado, total R$
  ↳ ${evento.totalCentavos / 100}`);
}
```

```
function notificarLogistica(evento: PedidoConfirmadoEvent) {  
  console.log(`[LOGÍSTICA] Preparar separação do pedido ${evento.pedidoId}`);  
}  
  
const pedido = new Pedido('p1', 15000);  
pedido.confirmar();  
  
for (const evento of pedido.liberarEventos()) {  
  enviarEmailConfirmacao(evento);  
  notificarLogistica(evento);  
}
```

Retorno esperado

```
[E-MAIL] Pedido p1 confirmado, total R$ 150  
[LOGÍSTICA] Preparar separação do pedido p1
```

Dica de Pleno

`Pedido.confirmar()` não importa nada de e-mail ou logística — só registra “isso aconteceu”. Esse desacoplamento permite adicionar um terceiro handler (ex: `atualizarMetricasDeVendas`) **sem tocar em uma linha da Aggregate Root**, só registrando um novo listener. É o Open/Closed Principle aplicado via eventos: aberto para extensão (novos handlers), fechado para modificação (a Entity que emite o evento não muda).

Por baixo dos panos: da chamada de função à mensageria

Por Baixo dos Panos

Em produção, Domain Events costumam evoluir de “chamada de função síncrona in-process” (como no exemplo acima) para **mensagens publicadas em um message broker** (RabbitMQ, Kafka) quando o handler precisa rodar em outro processo ou serviço — é exatamente essa evolução natural que conecta DDD a Event-Driven Architecture, tema central da Parte 3 (seção 3.4). A modelagem do evento em si (nome no passado, imutável, contendo só os dados relevantes do que aconteceu) não muda; o que muda é só o **mecanismo de entrega**: de uma chamada de método direta para uma fila assíncrona. É por isso que projetar bem os Domain Events desde o monólito facilita — e muito — uma futura extração para microsserviços.

Armadilha Comum

Um erro sutil e comum: disparar o Domain Event **antes** de a transação de banco que salva o novo estado ter sido efetivamente confirmada (commitada). Se o handler de e-mail dispara assim que `pedido.confirmar()` é chamado, mas a query que persiste esse novo status falha logo depois (ex: timeout de conexão), o cliente recebe um

e-mail de “pedido confirmado” para um pedido que **no banco continua pendente**. A prática correta é liberar e processar os eventos **depois** do commit da transação — o `@nestjs/cqrs` (seção 2.8) e a maioria dos ORMs oferecem hooks específicos de “publicar depois do commit” por esse motivo exato.

Cenário de Falha em Produção

Um handler de Domain Event dispara antes do commit em um sistema de assinaturas: o e-mail de “assinatura ativada” sai, mas a query que grava o novo status no banco falha por timeout de conexão logo em seguida. O cliente recebe o e-mail, tenta usar o produto, e recebe “acesso negado” — porque no banco a assinatura continua como pendente. O time de suporte recebe reclamações sem conseguir reproduzir o problema, porque em testes locais (sem sobrecarga de banco) a query quase nunca falha nesse ponto específico.

Dica de Produção

Se seu time reporta reclamações do tipo “recebi confirmação por e-mail mas o sistema mostra pendente”, a hipótese a validar primeiro é justamente essa: evento disparado antes do commit, não depois. Instrumente logs (ou trace spans) marcando explicitamente o momento do commit e o momento da publicação do evento — se em algum caso a publicação aparece **antes** do commit, você achou a causa raiz sem precisar adivinhar.

1.7 Bounded Contexts e Context Mapping

Já introduzimos a ideia de Bounded Context na seção 1.1: fronteiras dentro das quais um modelo (e sua linguagem) é consistente e válido. Um sistema real geralmente tem vários — Vendas, Catálogo, Cobrança, Logística — e a pergunta prática é: **como esses contextos se comunicam entre si?** Essa é a área de “Context Mapping”, e os padrões mais comuns são:

Padrão	O que significa	Quando aparece
Shared Kernel	Dois contextos compartilham deliberadamente parte do modelo (ex: um pacote comum de tipos)	Times próximos, mudanças coordenadas
Customer/Supplier	Um contexto (supplier) expõe uma API estável; outro (customer) consome e depende dela	Relação clara de dependência, com contrato negociado
Conformist	O contexto consumidor simplesmente aceita o modelo do fornecedor como está, sem tradução	Consumindo uma API externa (ex: gateway de pagamento) sem poder de negociação
Anticorruption Layer (ACL)	O contexto consumidor cria uma camada de tradução entre o modelo externo e o seu próprio modelo interno	Integrando com sistema legado ou externo cujo modelo você não quer que “vaze” para o seu domínio

O **Anticorruption Layer** é o mais relevante no dia a dia de quem trabalha com integrações: ele evita que o vocabulário e as estruturas de dados de um sistema externo (um ERP legado, uma API de terceiro) “vazem” para dentro do seu domínio limpo. Na prática, quem implementa uma ACL dentro de um projeto NestJS é o **Mapper** — a seção 2.6 (Parte 2) mostra a implementação completa desse padrão, incluindo o exemplo de tradução de um ERP legado; aqui basta reter o conceito: é o mesmo papel do Mapper que traduz um DTO HTTP, só que a “fonte externa” agora é um sistema de terceiro, não o corpo de uma requisição.

Cenário de Falha em Produção

Um time integra diretamente com a resposta bruta de uma API de pagamentos de terceiro, sem Anticorruption Layer — estruturas como `payment_status_cd: number (0, 1, 2, 3...)` são usadas diretamente em ifs espalhados por 15 arquivos do domínio interno. Quando o provedor de pagamento faz uma atualização de versão da API e **reordena os códigos numéricos** de status (mudança documentada, mas não amplamente lida), todos os 15 pontos precisam ser encontrados e corrigidos sob pressão, e pelo menos 2 são esquecidos — um deles fazendo pedidos com pagamento **recusado** serem tratados internamente como **aprovados** por algumas horas, até o erro ser detectado via reconciliação financeira.

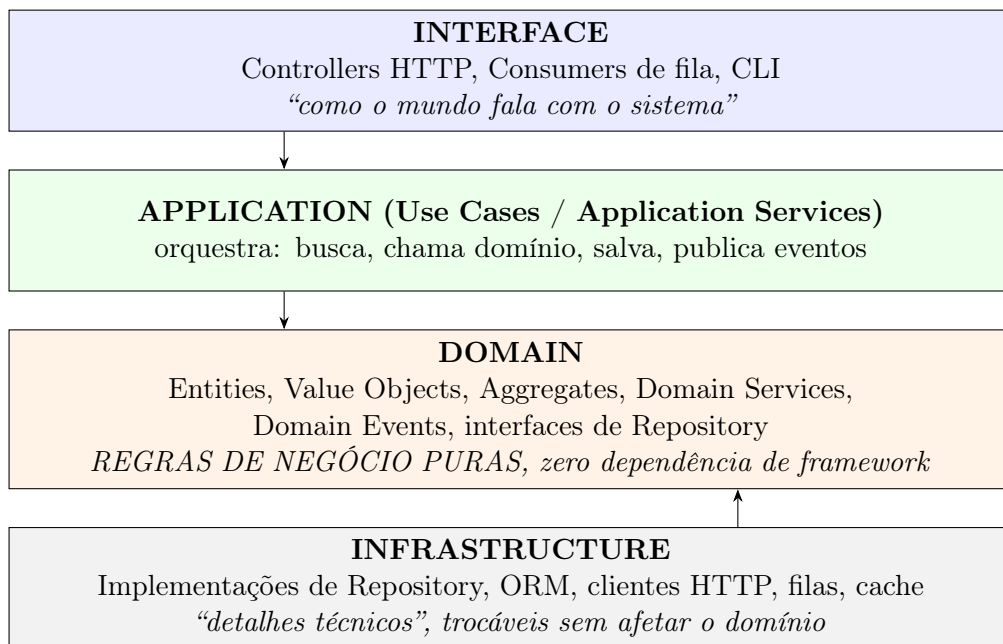
Dica de Produção

Uma régua simples para saber se você precisa de uma ACL: pergunte “se esse sistema externo mudar o formato da resposta amanhã, eu conserto em **um lugar** ou em quantos lugares o código usa esse formato diretamente?”. Se a resposta é “mais de um”, a integração já deveria ter (ou já deveria ganhar, com prioridade) uma camada de tradução única. Esse é também o ponto de instrumentação mais valioso: logar a versão/formato da resposta bruta recebida do fornecedor externo, para que uma

mudança silenciosa de contrato apareça nos logs antes de virar um incidente de negócio.

1.8 Camadas de uma aplicação DDD

Juntando os conceitos das seções 1.1 a 1.7, uma aplicação DDD tipicamente se organiza em 4 camadas — e essa organização é o que conecta DDD com Clean Architecture / Arquitetura Hexagonal (mesma ideia, vocabulários diferentes):



*Regra de dependência: INTERFACE → APPLICATION → DOMAIN.
INFRASTRUCTURE implementa interfaces definidas pelo DOMAIN (a seta sempre aponta para dentro).*

Dica de Pleno

A camada de **DOMAIN** é a única que deveria sobreviver, sem nenhuma alteração, a uma troca de framework HTTP (Express → Fastify), de ORM (TypeORM → Prisma), ou até de forma de entrega (API REST → mensageria). Se ao trocar o ORM você precisa alterar uma **Entity** ou um **Domain Service**, é sinal de que a camada de domínio, em algum ponto, “vazou” uma dependência de infraestrutura — o teste mais direto disso é: seu domínio importa algo de `@prisma/client`, `typeorm`, `express` ou `@nestjs/*` em algum arquivo? Se sim, a fronteira foi rompida. Se você tivesse que escolher uma única pergunta para auditar a saúde de DDD em um projeto existente, seria exatamente essa.

Desafio de Mentoria

Sua equipe herdou um sistema de 3 anos, um monólito com controllers Express que fazem query SQL direto (`db.query('SELECT ...')`) misturada com validação e regra de negócio no mesmo arquivo — nenhuma das 4 camadas existe hoje. O tech lead pede um plano de “migração gradual para DDD” que não pare o desenvolvimento de features novas por meses. Como Advogado do Diabo: você reescreveria tudo de uma vez (big bang) ou extrairia camada por camada, endpoint por endpoint (strangler fig — o mesmo padrão que a seção 3.1 usa para extrair microsserviços de um monólito)? Que critério você usaria para escolher **qual** dos 40 endpoints do sistema migrar primeiro — o mais simples (para “aprender fazendo” com baixo risco) ou o que concentra mais bugs de regra de negócio duplicada (onde o ganho de ter um domínio único e testável é maior)? Justifique considerando que o time tem 4 desenvolvedores e não pode parar de entregar.

Fim da Parte 1

Os sete conceitos desta parte formam uma cadeia contínua, não peças soltas: **Entity e Value Object** (1.2) são o vocabulário básico de modelagem; **Aggregate e Aggregate Root** (1.3) definem onde fica a fronteira de consistência transacional; **Repository** (1.4) abstrai a persistência para fora do domínio; **Domain Service e Application Service** (1.5) separam regra de negócio pura de orquestração de infraestrutura; **Domain Events** (1.6) permitem que a fronteira de consistência avise o resto do sistema sobre o que aconteceu, sem acoplamento direto; e **Bounded Context** (1.7) delimita onde cada modelo é válido.

Agora que temos nosso modelo de domínio bem definido — com suas Entities, Value Objects, Aggregates e a separação clara de camadas do diagrama acima —, a pergunta é: **como o implementamos de forma estruturada em um framework real?** É aqui que o NestJS entra: um framework opinativo, com Injeção de Dependência nativa, que acomoda essas 4 camadas dentro da sua própria convenção de módulos. A Parte 2 faz exatamente essa ponte, do modelo teórico ao código rodando.

Chapter 2

Parte 2 — Conectando DDD ao Código: NestJS

2.1 Módulos, Controllers e Providers: a espinha dorsal do Nest

NestJS é um framework Node.js opinativo, fortemente inspirado no Angular, construído sobre três blocos fundamentais que se repetem em **todo** projeto Nest:

- **Module (@Module):** a unidade de organização. Todo projeto Nest tem pelo menos um módulo raiz (AppModule), e projetos maiores se dividem em módulos de feature (PedidosModule, ClientesModule). Um módulo declara quais controllers expõe, quais providers oferece, e quais outros módulos importa.
- **Controller (@Controller):** recebe requisições HTTP (ou de outro transporte — seção 3.3) e delega para providers. Um controller **não deveria conter lógica de negócio** — é só a camada de tradução entre “requisição HTTP” e “chamada de método”.
- **Provider (@Injectable):** qualquer classe que pode ser injetada como dependência — services, repositories, factories. É aqui que a lógica de fato mora.

```
// npm install @nestjs/core @nestjs/common reflect-metadata rxjs
// npx ts-node main.ts (com tsconfig experimentalDecorators +
↳ emitDecoratorMetadata habilitados)

import { Module, Controller, Injectable, Get, Post, Body } from
↳ '@nestjs/common';
import { randomUUID } from 'crypto';

// PROVIDER - lógica de aplicação, injetável
@Injectable()
class PedidosService {
```

```
private pedidos: { id: string; total: number }[] = [];  
  
criar(total: number) {  
  const pedido = { id: randomUUID(), total };  
  this.pedidos.push(pedido);  
  return pedido;  
}  
  
listar() {  
  return this.pedidos;  
}  
}  
  
// CONTROLLER - só traduz HTTP para chamadas de método, zero lógica de negócio  
@Controller('pedidos')  
class PedidosController {  
  constructor(private readonly pedidosService: PedidosService) {} // DI via  
    ↔ construtor  
  
  @Post()  
  criar(@Body('total') total: number) {  
    return this.pedidosService.criar(total);  
  }  
  
  @Get()  
  listar() {  
    return this.pedidosService.listar();  
  }  
}  
  
// MODULE - amarra controller + provider  
@Module({  
  controllers: [PedidosController],  
  providers: [PedidosService],  
})  
class PedidosModule {}
```

Dica de Pleno

O Nest resolve `PedidosService` no construtor do `PedidosController` **sem** você escrever `new PedidosService()` em lugar nenhum — isso é o container de Injeção de Dependência do Nest funcionando via **metadata de tipos do TypeScript** (`emitDecoratorMetadata`). O framework lê, em tempo de execução, os tipos dos parâmetros do construtor (informação que o TypeScript emite como metadata quando compilado) para saber o que instanciar e injetar. É por isso que Nest exige decorators e uma configuração específica de `tsconfig.json` — sem isso, essa “mágica” de DI automática não teria como funcionar.

Por Baixo dos Panos

Por padrão, todo provider no Nest é um **singleton no escopo do módulo** — a mesma instância de `PedidosService` é reutilizada em **todas** as requisições, a menos que você declare explicitamente um escopo diferente (seção 2.2). Isso significa que qualquer estado guardado como propriedade de instância (`private pedidos: [...]` no exemplo acima, usado só para simplificar a demonstração) é **compartilhado entre requisições concorrentes de usuários diferentes** — um padrão perigoso em produção real, e o motivo pelo qual providers do Nest quase sempre devem ser **stateless**, delegando qualquer estado para um banco de dados, cache, ou outro armazenamento externo à instância do provider.

Cenário de Falha em Produção

Um `CarrinhoService` guarda o carrinho de compras atual como propriedade de instância (`private carrinhoAtual: Item[] = []`) “para simplificar”, sem perceber que o provider é singleton. Em produção, com múltiplos usuários simultâneos, o carrinho de um usuário **sobrescreve** o de outro, porque todos compartilham a mesma instância do service — um usuário finaliza a compra e vê itens de outro carrinho na tela de confirmação. O bug não aparece em testes locais (um dev testando sozinho nunca tem concorrência real), só se manifesta sob carga real com múltiplos usuários simultâneos, tornando-o particularmente perigoso de não pegar antes de produção.

Dica de Produção

Em code review de providers Nest, trate qualquer propriedade de instância mutável fora de um valor de configuração fixo (algo lido uma vez no **constructor** e nunca alterado depois) como suspeita — pergunte explicitamente “esse estado pode vazar entre requisições de usuários diferentes?”. Um teste de carga simples (com `autocannon` disparando requisições concorrentes que alteram estado por usuários diferentes) é a forma mais direta de expor esse tipo de bug antes do deploy, já que ele raramente aparece em testes sequenciais.

Quando (não) usar NestJS

O resto desta parte assume que o Nest é a escolha certa, mas essa escolha merece a mesma crítica que este material já aplica a CQRS (2.8) e a microsserviços (3.1) — NestJS **não** é a resposta certa para todo projeto Node, e a maturidade está em saber reconhecer os dois lados:

- **NestJS brilha quando:** o time é médio/grande e precisa de convenções compartilhadas (múltiplos devs tocando o mesmo código, sem uma “arquitetura pessoal” de cada um); o domínio é complexo o suficiente para se beneficiar de DI, módulos e das camadas da seção 2.7; o projeto vai viver anos e precisa ser onboardável por gente nova sem depender de decisões tribais; ou o time já vem do ecossistema Angular/DI e o vocabulário do Nest reduz a curva de aprendizado em vez de aumentá-la.

- **NestJS atrapalha quando:** é uma API pequena (poucos endpoints, sem regra de negócio complexa) onde o boilerplate de módulos/decorators supera, de longe, a lógica real — Express ou Fastify puro resolvem em menos código e menos indição; o time não tem familiaridade com decorators/DI do TypeScript e o tempo de aprendizado do framework compete com o prazo de entrega; ou o serviço roda como **função serverless** (AWS Lambda, Cloud Functions) com cold starts frequentes — o custo de bootstrap do container de DI do Nest a cada cold start é overhead mensurável que um handler simples de função não paga, especialmente em funções de baixa complexidade chamadas com baixa frequência.

Dica de Pleno

A pergunta prática antes de escolher Nest para um novo serviço: “esse serviço vai crescer em complexidade de domínio, ou vai continuar sendo um punhado de endpoints simples pela vida dele?”. Se a resposta é a segunda, o overhead de aprendizado e de estrutura do Nest é custo sem retorno correspondente — nesse caso, um Express/Fastify simples, estruturado com boas práticas manuais (as mesmas camadas da seção 1.8, sem framework nenhum ditando a forma), entrega o mesmo resultado com menos fricção.

2.2 Injeção de Dependência e Escopos de Provider

O Nest suporta três escopos (**Scope**) de provider, e escolher o errado é uma fonte real de bugs sutis ou de degradação de performance:

Escopo	Comportamento	Quando usar
<code>Scope.DEFAULT</code> (singleton)	Uma única instância para toda a aplicação, compartilhada entre todas as requisições	Padrão — a maioria dos services stateless
<code>Scope.REQUEST</code>	Uma nova instância criada a cada requisição HTTP	Quando o provider precisa de dados específicos da requisição (ex: usuário autenticado, tenant em multi-tenancy)
<code>Scope.TRANSIENT</code>	Uma nova instância a cada injeção (mesmo dentro da mesma requisição, se injetado em 2 lugares, são 2 instâncias)	Casos raros — providers que não podem compartilhar nenhum estado interno entre consumidores

```
// npm install @nestjs/core @nestjs/common
import { Injectable, Scope, Inject } from '@nestjs/common';
import { REQUEST } from '@nestjs/core';
```

```
// SINGLETON (padrão) - uma instância para a aplicação inteira, sem estado por
↪ requisição
@Injectable()
class LoggerService {
  log(mensagem: string) {
    console.log(`[LOG] ${mensagem}`);
  }
}

// REQUEST-SCOPED - nova instância a cada requisição, pode acessar o objeto
↪ `request`
@Injectable({ scope: Scope.REQUEST })
class TenantContextService {
  constructor(@Inject(REQUEST) private readonly request: any) {}

  obterTenantId(): string {
    return this.request.headers['x-tenant-id']; // dado específico DESSA
    ↪ requisição
  }
}
```

Dica de Pleno

`Scope.REQUEST` resolve um problema real (acessar dados da requisição atual, como um tenant ID ou usuário autenticado, de forma “limpa” via DI em vez de passar esses dados manualmente por todos os métodos), mas tem um custo: **qualquer provider que dependa, direta ou indiretamente, de um provider REQUEST-scoped também vira REQUEST-scoped**, propagando a recriação em cascata. Se um `TenantContextService` request-scoped é injetado em um `PedidosService` que, por sua vez, é usado por 10 outros services, todos os 10 passam a ser recriados a cada requisição — perdendo o benefício de singleton (reuso de instância) na árvore inteira de dependências.

Armadilha Comum

Usar `Scope.REQUEST` “porque parece mais seguro” em providers que não precisam de dado nenhum da requisição é um erro comum de quem está migrando de um mindset onde toda instância é criada por chamada. Isso adiciona overhead real de performance (uma nova árvore de instâncias construída a cada requisição, em vez de reutilizar instâncias já resolvidas) sem nenhum benefício, se o provider não acessa `REQUEST` em lugar nenhum. Use `Scope.DEFAULT` (o padrão, nem precisa declarar) sempre que possível, e reserve `Scope.REQUEST` para os poucos providers que **de fato** precisam de dado específico da requisição atual.

Cenário de Falha em Produção

Um sistema multi-tenant usa `Scope.REQUEST` amplamente — não só no `TenantContextService`, mas em cascata em quase todos os services da aplicação (porque um deles injeta o outro, que injeta o outro, e assim por diante). Sob carga

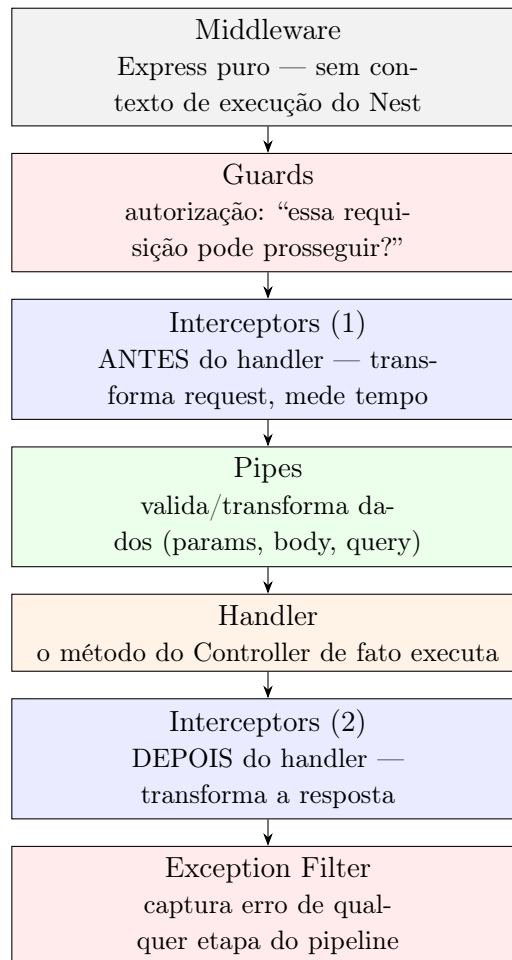
de produção (Black Friday, tráfego 10x o normal), a taxa de criação de objetos (garbage) dispara, aumentando drasticamente a frequência e o tempo de pausas do Garbage Collector do V8 — a API começa a apresentar picos de latência p99 correlacionados diretamente com o volume de requisições, mesmo com CPU aparentemente disponível, um sintoma clássico de pressão de GC que só é diagnosticado depois de um profiling de heap revelar milhares de instâncias de service de curtíssima duração sendo criadas e descartadas por segundo.

Dica de Produção

Uma alternativa mais barata a `Scope.REQUEST`, para o caso comum de “preciso saber quem é o usuário/tenant atual”, é usar `AsyncLocalStorage` (nativo do Node.js) para propagar esse contexto sem recriar toda a árvore de DI a cada requisição — o Nest mantém providers singleton, e o dado da requisição é lido de um contexto assíncrono ambiente em vez de injeção de escopo. Se um profiling de produção aponta alta taxa de alocação de objetos correlacionada com o número de providers `REQUEST-scoped` na aplicação, essa é a refatoração de maior retorno.

2.3 Pipes, Guards, Interceptors e Exception Filters: o pipeline de uma requisição

Toda requisição HTTP no Nest passa por um pipeline de componentes especializados, sempre **na mesma ordem**, e saber essa ordem de cor é o que diferencia quem “usa decorators porque viu em um tutorial” de quem entende o mecanismo:



```

// npm install @nestjs/common class-validator class-transformer
import {
  Injectable, CanActivate, ExecutionContext, PipeTransform, ArgumentMetadata,
  BadRequestException, NestInterceptor, CallHandler, ExceptionFilter, Catch,
  ⇨ ArgumentsHost,
} from '@nestjs/common';
import { Observable, tap } from 'rxjs';

// GUARD - decide se a requisição PASSA ou não (autorização)
@Injectable()
class ApenasAdminGuard implements CanActivate {
  canActivate(contexto: ExecutionContext): boolean {
    const request = contexto.switchToHttp().getRequest();
    return request.headers['x-role'] === 'admin'; // true = passa, false = 403
    ⇨ automático
  }
}

// PIPE - valida/transforma dados de entrada ANTES de chegar no handler
@Injectable()
class ParseIdPositivoPipe implements PipeTransform {
  transform(valor: any, metadata: ArgumentMetadata) {
    const numero = Number(valor);
  }
}

```

```

    if (Number.isNaN(numero) || numero <= 0) {
        throw new BadRequestException(`${metadata.data} deve ser um número
        ↪ positivo`);
    }
    return numero;
}
}

// INTERCEPTOR - envolve a execução do handler (útil para logging, cache,
↪ transformação de resposta)
@Injectable()
class LogTempoInterceptor implements NestInterceptor {
    intercept(contexto: ExecutionContext, next: CallHandler): Observable<any> {
        const inicio = Date.now();
        return next.handle().pipe(
            tap(() => console.log(`[TEMPO] ${Date.now() - inicio}ms`)), // roda DEPOIS
            ↪ do handler
        );
    }
}

// EXCEPTION FILTER - captura erros e formata a resposta de forma consistente
@Catch(BadRequestException)
class ValidacaoExceptionHandler implements ExceptionFilter {
    catch(excecao: BadRequestException, host: ArgumentsHost) {
        const response = host.switchToHttp().getResponse();
        response.status(400).json({ erro: 'validacao_falhou', detalhe:
        ↪ excecao.message });
    }
}

```

Dica de Pleno

A pergunta clássica de entrevista “qual a diferença entre Guard e Interceptor, já que os dois rodam antes do handler?” tem uma resposta precisa: **Guard responde uma pergunta binária de autorização** (pode ou não pode passar — e retornar false já produz automaticamente um 403 Forbidden, sem você escrever isso); **Interceptor participa do fluxo de dados** (pode transformar a requisição, ler/alterar a resposta depois que o handler roda, medir tempo, implementar cache). Um Guard nunca “vê” a resposta; um Interceptor vê os dois lados (antes E depois do handler), porque ele envolve a chamada com um Observable RxJS.

Armadilha Comum

Colocar lógica de autorização (checagem de permissão) dentro de um Interceptor, em vez de um Guard, é um erro comum que “funciona” mas quebra a separação de responsabilidades e, pior, pode custar mais caro em performance: um Interceptor mal escrito que lança exceção de autorização só depois de já ter iniciado efeitos colaterais (ex: já ter aberto uma transação de banco, ou já ter incrementado uma métrica de “requisição recebida”) gera trabalho desperdiçado e, em alguns casos,

estado inconsistente — porque a requisição não deveria ter chegado tão longe no pipeline. Guards existem exatamente para cortar a requisição **antes** de qualquer trabalho começar.

Cenário de Falha em Produção

Um endpoint de exclusão de conta (DELETE /contas/:id) implementa a checagem “só o próprio dono ou um admin pode excluir” dentro de um Interceptor (rodando **depois** de Guards genéricos de autenticação, mas antes do handler). Um refactor futuro adiciona um novo Interceptor de auditoria **antes** desse Interceptor de autorização na ordem de registro do módulo — como múltiplos interceptors rodam na ordem em que são declarados, o Interceptor de auditoria passa a registrar “tentativa de exclusão” no log de auditoria **mesmo para tentativas que serão barradas segundos depois** pelo interceptor de autorização, poluindo os logs de auditoria com falsos positivos de tentativas de exclusão não autorizadas, gerando alertas de segurança recorrentes que o time de segurança perde tempo investigando.

Dica de Produção

Regra prática consolidada pela comunidade Nest: autorização (quem pode fazer isso?) é sempre responsabilidade de **Guard**; transformação de request/response, medição, cache e logging são responsabilidade de **Interceptor**. Ao revisar um PR que adiciona um novo Interceptor, pergunte explicitamente “esse interceptor decide se algo é permitido, ou só observa/transforma o que já foi decidido que vai acontecer?” — se é a primeira opção, ele deveria ser um Guard.

Desafio de Mentoria

Você precisa implementar rate limiting por usuário (não mais que 100 requisições por minuto) em uma API Nest. Um colega sugere implementar isso como Guard (RateLimitGuard, retornando **false** quando o limite estoura); outro sugere implementar como Interceptor (interceptando e lançando TooManyRequestsException quando o limite estoura). Tecnicamente os dois “funcionam”. Como Advogado do Diabo, argumente pela escolha correta usando a distinção conceitual desta seção — e explique por que a resposta muda se o rate limit precisa **também** adicionar headers X-RateLimit-Remaining na resposta de requisições que passam pelo limite (não só bloquear as que estouram).

2.4 DTOs: a fronteira validada

O que é, para que serve

Um **DTO** (Data Transfer Object) é um objeto simples, sem comportamento de negócio, cuja única responsabilidade é **transportar dados através de uma fronteira** — tipicamente entre o mundo externo (o corpo de uma requisição HTTP) e a aplicação. Ele é

a “fôrma”: define exatamente que formato de dado é aceito naquela fronteira, sem saber nada sobre regras de domínio.

No NestJS, o par mais usado em produção para validar essa fronteira é o `ValidationPipe` embutido combinado com uma classe DTO decorada com `class-validator` — o Pipe é o mecanismo genérico apresentado na seção 2.3; o `ValidationPipe` é a implementação oficial mais usada desse mecanismo:

```
// npm install class-validator class-transformer
import { IsString, IsInt, Min, ValidateNested, ArrayMinSize } from
  ↳ 'class-validator';
import { Type } from 'class-transformer';
import { Controller, Post, Body } from '@nestjs/common';

class ItemPedidoDto {
  @IsString()
  produtoId!: string;

  @IsInt()
  @Min(1)
  quantidade!: number;
}

// DTO - a FRONTEIRA onde dado bruto e não confiável vira dado validado e tipado
class CriarPedidoDto {
  @ValidateNested({ each: true })
  @Type(() => ItemPedidoDto)
  @ArrayMinSize(1)
  itens!: ItemPedidoDto[];
}

@Controller('pedidos')
class PedidosController {
  @Post()
  criar(@Body() dto: CriarPedidoDto) {
    // ao chegar aqui, dto JÁ FOI validado - o handler não precisa checar nada
    return { itensRecebidos: dto.itens.length };
  }
}

// main.ts - registra o ValidationPipe GLOBALMENTE, para toda a aplicação
// app.useGlobalPipes(new ValidationPipe({
//   whitelist: true,           // remove campos que NÃO estão no DTO (protege
//   ↳ contra mass assignment)
//   forbidNonWhitelisted: true, // rejeita a requisição, em vez de só ignorar,
//   ↳ se vier campo extra
//   transform: true,          // converte o JSON bruto na INSTÂNCIA da classe
//   ↳ DTO
// }));
```

Dica de Pleno

Essa dupla `ValidationPipe` + DTO é a materialização, dentro do Nest, do mesmo princípio “**Parse, don’t validate**” já apresentado na seção 1.2 sobre Value Objects — o `CriarPedidoDto` é a fronteira que garante que, a partir do momento em que o handler executa, os dados **já são válidos por construção**. A opção `whitelist: true` merece atenção especial: sem ela, campos extras enviados no corpo da requisição (como um `role: 'admin'` que o cliente não deveria poder setar) passam despercebidos para dentro do objeto — uma classe de vulnerabilidade conhecida como **mass assignment**.

Quando usar, quando NÃO usar

Use **DTO** em toda fronteira de entrada de dado não confiável: corpo de requisição HTTP, payload de mensageria, resposta de uma API externa antes de tratá-la. **Não** transforme um DTO em um objeto de domínio disfarçado — um erro comum é adicionar métodos de regra de negócio a uma classe DTO (`criarPedidoDto.calcularDesconto()`), misturando a responsabilidade de “formato de dado na borda” com “regra de negócio”, que pertence à Entity ou ao Domain Service (seção 1.5). Se você percebe um DTO crescendo métodos de comportamento, é sinal de que uma Entity ou Value Object deveria existir e o DTO deveria só alimentá-la — o que nos leva à seção 2.6 (Mappers).

Armadilha Comum

Esquecer `whitelist: true` (ou não registrar `ValidationPipe` globalmente, aplicando-o só em alguns endpoints “que pareciam precisar”) é o erro mais caro dessa dupla. Um endpoint de atualização de perfil sem `whitelist` pode permitir que um cliente malicioso envie campos extras não previstos no DTO — se o service depois faz um `Object.assign` ingênuo do corpo bruto da requisição em vez de só dos campos do DTO validado, um usuário comum consegue definir campos como `isAdmin: true` em si mesmo, mesmo sem esse campo estar documentado ou esperado pela API.

Cenário de Falha em Produção

Um endpoint `PATCH /usuarios/:id` aceita um DTO `AtualizarUsuarioDto` sem `whitelist: true` no `ValidationPipe` global (configurado só para outros módulos). O service, por conveniência, faz `this.repo.update(id, dtoRecebido)` passando o DTO inteiro direto para o ORM, sem selecionar campos. Um usuário descobre, testando a API manualmente, que enviar `{ nome: "Ana", isAdmin: true }` no corpo — mesmo `isAdmin` não estando documentado nem existindo no DTO oficial — resulta em `isAdmin: true` sendo persistido, porque o ORM aceita qualquer chave presente no objeto JavaScript. O usuário vira administrador da própria conta sem nenhuma autorização.

Dica de Produção

Trate `whitelist: true` e `forbidNonWhitelisted: true` como configuração obrigatória, não opcional, no `ValidationPipe` global de todo projeto Nest — e nunca passe o DTO inteiro direto para uma operação de escrita do ORM (`update(id, dto)`); selecione explicitamente os campos que aquela operação tem permissão de alterar. Em auditoria de segurança, qualquer `Object.assign(entidade, dtoOuBodyBruto)` ou equivalente é prioridade de revisão imediata — é a assinatura clássica de mass assignment.

2.5 Decorators customizados e metadata com Reflector

Decorators são a linguagem visual do Nest — `@Controller`, `@Injectable`, `@Get` já vêm prontos, mas criar decorators customizados é o que permite expressar conceitos do **seu próprio domínio/aplicação** de forma declarativa, reduzindo boilerplate repetido em dezenas de controllers.

```
// npm install @nestjs/common @nestjs/core
import { SetMetadata, createParamDecorator, ExecutionContext, UseGuards,
  ↳ CanActivate, Injectable, Reflector } from '@nestjs/common';

// DECORATOR DE METADATA - anexa informação a um handler, para ser lida depois
↳ por um Guard
const PERMISSAO_KEY = 'permissao_requerida';
export const RequerPermissao = (permissao: string) =>
  ↳ SetMetadata(PERMISSAO_KEY, permissao);

// DECORATOR DE PARÂMETRO - extrai um dado do request de forma reutilizável e
↳ tipada
export const UsuarioAtual = createParamDecorator((_dado: unknown, contexto:
  ↳ ExecutionContext) => {
  const request = contexto.switchToHttp().getRequest();
  return request.usuario; // populado antes por algum middleware/guard de
  ↳ autenticação
});

// GUARD que Lê a metadata anexada pelo decorator customizado, via Reflector
@Injectable()
class PermissaoGuard implements CanActivate {
  constructor(private readonly reflector: Reflector) {}

  canActivate(contexto: ExecutionContext): boolean {
    const permissaoRequerida = this.reflector.get<string>(PERMISSAO_KEY,
      ↳ contexto.getHandler());
    if (!permissaoRequerida) return true; // rota sem @RequerPermissao - sem
      ↳ restrição
    const request = contexto.switchToHttp().getRequest();
```

```
    return request.usuario?.permissoes?.includes(permisaoRequerida) ?? false;
  }
}

// USO - declarativo, lê como uma frase
// @UseGuards(PermissaoGuard)
// @RequerPermissao('pedidos:cancelar')
// @Delete(':id')
// cancelar(@Param('id') id: string, @UsuarioAtual() usuario: any) { ... }
```

Dica de Pleno

SetMetadata + Reflector é o mecanismo genérico por trás de praticamente **todos** os decorators declarativos do ecossistema Nest, incluindo os de bibliotecas de terceiros (**@Roles()** do exemplo de RBAC oficial, **@Public()** de bibliotecas de auth, **@CacheTTL()** de módulos de cache). Entender esse par (**SetMetadata** anexa, **Reflector.get** lê) é entender como estender o Nest com sua própria linguagem declarativa, sem precisar de nenhuma mágica adicional — é só metadata do Reflect API do JavaScript, exposta de forma ergonômica.

Por Baixo dos Panos

SetMetadata e **Reflector** são uma camada de conveniência sobre a **Reflect Metadata API** (**reflect-metadata**, o mesmo pacote que habilita a injeção de dependência automática da seção 2.1). Cada decorator anexa um par chave-valor ao objeto da classe/método usando **Reflect.defineMetadata** por baixo dos panos; **Reflector.get** faz **Reflect.getMetadata** no alvo certo (handler do método, ou classe do controller, dependendo de onde você aplicou o decorator). Não há nenhuma mágica de framework aqui além dessa API padrão do TC39 Stage 3 (decorators) combinada com a implementação de metadata reflection que o TypeScript historicamente usa.

Cenário de Falha em Produção

Um decorator customizado **@RequerPermissao** é aplicado em um método de controller, mas o **PermissaoGuard** correspondente é registrado **apenas no módulo daquele controller específico** (**providers: [PermissaoGuard]**, sem ser global) — meses depois, um novo controller em outro módulo usa o mesmo decorator **@RequerPermissao('financeiro:ver')**, mas esquece de aplicar **@UseGuards(PermissaoGuard)** (só o decorator de metadata sozinho **não faz nada** — ele só anexa um dado, não executa checagem nenhuma). O endpoint fica exposto sem controle de permissão nenhum, apesar de “parecer” protegido porque tem o decorator visualmente presente — um caso clássico de segurança quebrada por composição incompleta, que passa despercebido em review porque o decorator “está lá”.

Dica de Produção

Sempre que um decorator de metadata (como `@RequerPermissao`) depende de um Guard específico para ter efeito, prefira registrar esse Guard **globalmente** (via `APP_GUARD` no módulo raiz) em vez de módulo a módulo — isso garante que o decorator **nunca** é “silenciosamente ignorado” por falta do Guard correspondente estar registrado. Se optar por registro local mesmo assim, escreva um teste de integração que varra todos os controllers da aplicação e falhe explicitamente se encontrar `@RequerPermissao` sem o Guard correspondente aplicado — transformando um erro silencioso de segurança em uma falha de CI, detectável antes do deploy.

2.6 Mappers e Anticorruption Layer: a tradução entre DTO e domínio

DTO é a fôrma, Mapper é a tradução

Se o DTO (seção 2.4) define a **estrutura** do dado que entra na fronteira, o **Mapper** é a **lógica que traduz** esse dado — convertendo o DTO bruto e validado em Value Objects ou Entities do domínio, ou traduzindo de volta uma Entity de domínio para um formato de saída (response). Onde o DTO só *descreve a forma* (“um pedido precisa ter uma lista de itens, cada um com `produtoId` e `quantidade`”), o Mapper *executa o ato* de pegar aquele dado bruto e despejá-lo dentro dos objetos internos do domínio, no vocabulário e no formato que o domínio espera.

```
// application/mappers/criar-pedido.mapper.ts

// DTO (a fôrma) - já validado pelo ValidationPipe antes de chegar aqui
class CriarPedidoDto {
  itens!: { produtoId: string; quantidade: number; precoCentavos: number }[];
}

// Objetos de DOMÍNIO - o que o Mapper produz
class Dinheiro {
  private constructor(private readonly centavos: number) {}
  static deCentavos(centavos: number): Dinheiro {
    if (centavos <= 0) throw new Error('Preço deve ser positivo');
    return new Dinheiro(centavos);
  }
  get valor() { return this.centavos; }
}

class Pedido {
  private itens: { produtoId: string; quantidade: number; preco: Dinheiro }[] =
    [];
  constructor(public readonly id: string) {}
  adicionarItem(produtoId: string, quantidade: number, preco: Dinheiro) {
```

```
        this.itens.push({ produtoId, quantidade, preco });
    }
}

// MAPPER - a TRADUÇÃO entre a fôrma (DTO) e o domínio (Entity + Value Objects)
class CriarPedidoMapper {
    static paraDominio(dto: CriarPedidoDto, novoId: string): Pedido {
        const pedido = new Pedido(novoId);
        for (const item of dto.itens) {
            // aqui, e SÓ aqui, o dado bruto do DTO vira um Value Object validado do
            ↪ domínio
            pedido.adicionarItem(item.produtoId, item.quantidade,
                ↪ Dinheiro.deCentavos(item.precoCentavos));
        }
        return pedido;
    }
}

const dto: CriarPedidoDto = { itens: [{ produtoId: 'p1', quantidade: 2,
    ↪ precoCentavos: 5000 }] };
const pedido = CriarPedidoMapper.paraDominio(dto, 'pedido-123');
console.log('Pedido criado a partir do DTO:', pedido.id);
```

Retorno esperado

Pedido criado a partir do DTO: pedido-123

Dica de Pleno

Repare a divisão de responsabilidade: `CriarPedidoDto` só declara **a forma** do dado esperado (o que o `ValidationPipe` usa para validar na borda); `CriarPedidoMapper.paraDominio` executa **a tradução** — pega o dado já validado e monta os objetos de domínio reais, incluindo a conversão de um `number` bruto (`precoCentavos`) em um `Value Object` validado (`Dinheiro.deCentavos`). DTO é a estrutura; Mapper é o comportamento de conversão. Um sem o outro é incompleto: um DTO sem Mapper deixa o dado bruto vazar para dentro do domínio; um Mapper sem DTO não tem garantia nenhuma de que o dado de entrada já está no formato esperado antes de tentar traduzir.

Mapper como Anticorruption Layer: protegendo o domínio de vocabulário externo

O mesmo papel do Mapper se aplica, com ainda mais força, quando o dado de entrada não vem de um cliente HTTP bem-comportado, mas de um **sistema externo cujo vocabulário você não controla** — um ERP legado, uma API de terceiro. Nesse caso, o Mapper se torna o que a seção 1.7 chamou de **Anticorruption Layer (ACL)**: a única camada que “conhece” o formato estranho do sistema externo, traduzindo-o para a

linguagem limpa do seu próprio domínio antes que esse vocabulário externo “vaze” para dentro do resto do sistema.

```
// npx ts-node anticorruption-layer.ts

// Modelo EXTERNO (resposta de uma API de ERP legado, fora do seu controle)
interface ErpLegadoClienteResponse {
  COD_CLI: string;
  NM_CLIENTE: string;
  FLG_ATIVO: '0' | '1'; // '1' = ativo, no vocabulário estranho do sistema
  ↪ legado
}

// Modelo do SEU DOMÍNIO - limpo, na SUA linguagem ubíqua
class Cliente {
  constructor(
    public readonly id: string,
    public readonly nome: string,
    public readonly ativo: boolean,
  ) {}
}

// MAPPER / ANTICORRUPTION LAYER - único lugar que conhece o formato feio do
↪ legado
class ErpClienteMapper {
  static paraDominio(respostaErp: ErpLegadoClienteResponse): Cliente {
    return new Cliente(
      respostaErp.COD_CLI,
      respostaErp.NM_CLIENTE.trim(),
      respostaErp.FLG_ATIVO === '1', // traduz o vocabulário do legado para o do
      ↪ domínio
    );
  }
}

const respostaCrua: ErpLegadoClienteResponse = { COD_CLI: '00042', NM_CLIENTE:
↪ 'Ana Souza ', FLG_ATIVO: '1' };
const cliente = ErpClienteMapper.paraDominio(respostaCrua);
console.log('Traduzido:', cliente);
```

Retorno esperado

```
Traduzido: Cliente { id: '00042', nome: 'Ana Souza', ativo: true }
```

Dica de Pleno

Sem o `ErpClienteMapper`, o campo `FLG_ATIVO: '0' | '1'` (herança de algum sistema antigo) vazaria para dentro do seu domínio, e você acabaria escrevendo

if (cliente.FLG_ATIVO === '1') espalhado pelo código novo — contaminando sua linguagem ubíqua com o vocabulário de um sistema que você nem controla. O Mapper/ACL é a fronteira que absorve esse “ruído” em um único lugar, exatamente como o `CriarPedidoMapper` do exemplo anterior absorve a tradução de um DTO HTTP comum — a diferença é só a origem do dado bruto (API externa vs corpo de requisição), o papel do Mapper é idêntico nos dois casos.

A diferença cristalizada em uma frase

DTO é a fôrma — a estrutura que descreve que formato de dado é aceito numa fronteira. Mapper é a tradução — a lógica que pega o dado naquela fôrma e o converte para o formato interno que o domínio entende, validando e traduzindo vocabulário no processo. Um DTO nunca deveria conter lógica de conversão; um Mapper nunca deveria ser o lugar onde a validação estrutural básica acontece (isso é papel do `ValidationPipe` + DTO, antes do Mapper ser sequer chamado). Cada um faz uma coisa, e a fronteira entre os dois é o que mantém o domínio protegido tanto de dado malformatado quanto de vocabulário externo.

Armadilha Comum

Pular o Mapper e passar o DTO diretamente para dentro do domínio (ex: um `UseCase` que recebe `CriarPedidoDto` e o usa como se fosse a `Entity`) é um atalho tentador que quebra o isolamento do domínio silenciosamente. Funciona enquanto o formato do DTO e o formato da `Entity` coincidem — e quebra, de forma sutil, no dia em que a `Entity` precisa de um `Value Object` (`Dinheiro`) onde o DTO só tem um `number` cru, ou quando o DTO tem um campo opcional que a `Entity` não deveria aceitar como opcional. Nesse ponto, alguém vai “ajeitar” a `Entity` para aceitar o formato do DTO, em vez de criar o Mapper que deveria ter existido desde o início — e o domínio começa a carregar decisões de formato da API HTTP.

Dica de Produção

Uma regra prática de revisão de código: se um `UseCase` ou `Application Service` recebe um parâmetro do tipo `AlgumDto` diretamente (em vez de receber já os tipos de domínio, como `Pedido` ou `Dinheiro`), pergunte “onde está o Mapper que faz essa conversão?”. Se a resposta é “não tem, o DTO é usado direto”, esse é o ponto exato onde o vocabulário e as constraints da camada HTTP vão, cedo ou tarde, vazar para dentro da lógica de negócio — o mesmo tipo de vazamento que a `Anticorruption Layer` existe para evitar, só que dessa vez a “fonte externa” é a sua própria API, não um sistema de terceiro.

2.7 Aplicação Prática: estruturando DDD dentro de um projeto NestJS

Juntando os conceitos das seções anteriores, a pergunta prática é: em um projeto Nest real, estruturado por camadas de DDD, **onde fisicamente** ficam o DTO, o Mapper, a Entity, o Aggregate e o Domain Event? A convenção mais adotada na comunidade é: **as camadas de DDD viram subpastas dentro de cada módulo de feature do Nest**, não módulos Nest separados.

```
src/
  pedidos/                                <- módulo de feature do Nest (Bounded
    ↳ Context)
    domain/                              <- CAMADA DOMAIN - zero import de @nestjs/*
      pedido.entity.ts                   <- Aggregate Root (contém os ItemPedido
        ↳ internos)
      item-pedido.entity.ts              <- Entity interna ao Aggregate (não exposta)
      dinheiro.value-object.ts           <- Value Object
      pedido-confirmado.event.ts         <- Domain Event
      pedido.repository.ts               <- INTERFACE do repositório (porta)
    application/                         <- CAMADA APPLICATION
      criar-pedido.use-case.ts           <- orquestra Mapper + domínio + repo +
        ↳ eventos
    mappers/
      criar-pedido.mapper.ts             <- MAPPER: DTO -> domínio (seção 2.6)
    infrastructure/                     <- CAMADA INFRASTRUCTURE
      pedido.repository.prisma.ts        <- implementação concreta do repositório
      event-bus.rabbitmq.ts              <- publica Domain Events como mensagens
        ↳ (Parte 3)
    interface/                           <- CAMADA INTERFACE - aqui SIM entra
      ↳ @nestjs/*
      dto/
        criar-pedido.dto.ts              <- DTO: fôrma validada pelo ValidationPipe
          ↳ (seção 2.4)
        pedidos.controller.ts
      pedidos.module.ts                   <- amarra tudo via DI do Nest
```

```
// pedidos.module.ts - a "cola" que conecta DDD puro ao container de DI do Nest
import { Module } from '@nestjs/common';
import { PedidosController } from './interface/pedidos.controller';
import { CriarPedidoUseCase } from './application/criar-pedido.use-case';
import { PedidoRepositoryPrisma } from
↳ './infrastructure/pedido.repository.prisma';

@Module({
  controllers: [PedidosController],
  providers: [
    CriarPedidoUseCase,
```

```

    { provide: 'PedidoRepository', useClass: PedidoRepositoryPrisma }, // liga a
      ↪ INTERFACE à IMPLEMENTAÇÃO
  ],
})
export class PedidosModule {}

```

O ponto onde DTO, Mapper, Entity e Event se encontram, de fato, é o Use Case — o exemplo abaixo mostra o fluxo completo, do dado bruto validado até os eventos liberados:

```

// application/criar-pedido.use-case.ts - o ponto onde DTO, Mapper, Entity e
↪ Event se encontram
import { Injectable, Inject } from '@nestjs/common';
import { CriarPedidoDto } from '../interface/dto/criar-pedido.dto';
import { CriarPedidoMapper } from '../mappers/criar-pedido.mapper';
import type { PedidoRepository } from '../domain/pedido.repository';
import type { EventBus } from '../infrastructure/event-bus.rabbitmq';

@Injectable()
export class CriarPedidoUseCase {
  constructor(
    @Inject('PedidoRepository') private readonly repositório: PedidoRepository,
    @Inject('EventBus') private readonly eventBus: EventBus,
  ) {}

  async executar(dto: CriarPedidoDto): Promise<string> {
    // 1. MAPPER: traduz o DTO validado em objetos de domínio (Entity + Value
    ↪ Objects)
    const pedido = CriarPedidoMapper.paraDominio(dto, crypto.randomUUID());

    // 2. DOMÍNIO: a Aggregate Root confirma, validando suas próprias
    ↪ invariantes
    pedido.confirmar();

    // 3. INFRAESTRUTURA: persiste através da interface de Repository
    await this.repositório.salvar(pedido);

    // 4. EVENTOS: publica os Domain Events acumulados para quem quiser reagir
    for (const evento of pedido.liberarEventos()) {
      await this.eventBus.publicar(evento);
    }

    return pedido.id;
  }
}

```

Dica de Pleno

Repare a ordem exata em que os conceitos deste guia se encontram nesse único método: o **DTO** (2.4) chega já validado pelo **ValidationPipe** (interceptado antes mesmo do Controller chamar o Use Case); o **Mapper** (2.6) traduz esse DTO em objetos de **domínio** (Entities e Value Objects, 1.2); a **Aggregate Root** (1.3) valida suas próprias invariantes; o **Repository** (1.4) persiste; e os **Domain Events** (1.6) acumulados são liberados só depois — nunca antes — da persistência ter acontecido (a mesma armadilha discutida na seção 1.6). Se você consegue apontar, no seu projeto atual, exatamente esses cinco passos em algum Use Case central, sua aplicação de DDD está estruturalmente saudável.

O token 'PedidoRepository' (string) usado em `@Inject` e `provide` merece atenção: é o que permite ao Application Service depender só da **interface** `PedidoRepository` (que não existe em tempo de execução — interfaces do TypeScript são apagadas na compilação, então não podem ser usadas como token de DI diretamente). Esse é o motivo pelo qual DI baseada em interfaces em TypeScript sempre precisa desse “token” intermediário (string, `Symbol`, ou uma classe abstrata) — o container de DI do Nest resolve por esse token, não pelo tipo TypeScript, que não existe mais depois de compilado para JavaScript puro.

Armadilha Comum

Importar `PrismaClient` (ou qualquer coisa de `@prisma/client`) dentro da pasta `domain/` é a violação mais comum (e mais fácil de detectar em review) dessa estrutura — geralmente acontece “só para tipar um campo mais rápido” (`Prisma.PedidoCreateInput`), mas isso acopla a Entity de domínio ao schema do ORM, quebrando exatamente o isolamento que a arquitetura em camadas existe para garantir. Da mesma forma, colocar lógica de Mapper **dentro do Controller** (fazendo a tradução DTO→domínio inline, junto com o roteamento HTTP) mistura duas responsabilidades que deveriam ficar em arquivos separados e testáveis isoladamente — o Mapper deveria ser testável sem subir nenhuma parte do Nest, exatamente como as Entities da seção 1.2.

Cenário de Falha em Produção

Um projeto que começou disciplinado, com `domain/` isolado, foi cedendo pressão de prazo: um desenvolvedor sob deadline importa `@nestjs/common` dentro de uma Entity de domínio só para usar `Logger` (conveniente, já está instalado). Seis meses depois, o time decide extrair esse Bounded Context para um microserviço separado, fora do Nest (um worker rodando em outra tecnologia, por exemplo — o cenário que a Parte 3 explora em detalhe), e descobre que a “camada de domínio pura” na verdade tem 12 pontos de acoplamento ao framework Nest acumulados ao longo do tempo — a extração planejada para “um dia de trabalho” (já que a lógica de negócio estava teoricamente isolada) vira duas semanas de desacoplamento retroativo.

Dica de Produção

Configure uma regra de lint (`eslint-plugin-boundaries` ou equivalente) que falhe o CI se `domain/` importar qualquer coisa de `infrastructure/`, `interface/` ou de

pacotes como @nestjs/*, @prisma/client, typeorm. O investimento de configurar essa checagem automatizada uma vez é ordens de magnitude menor que o custo de descobrir, meses depois, que “domínio puro” na verdade acumulou acoplamentos de framework ao longo do tempo — especialmente se um dia esse Bounded Context precisar ser extraído para um microsserviço separado, o assunto da Parte 3 deste guia.

2.8 CQRS com @nestjs/cqrs

CQRS (Command Query Responsibility Segregation) é o padrão de separar explicitamente operações que **mudam estado** (Commands) de operações que **leem estado** (Queries), cada uma com seu próprio caminho de código. O pacote oficial @nestjs/cqrs formaliza isso com CommandBus, QueryBus e EventBus (a mesma ideia de Domain Events da seção 1.6, agora com infraestrutura de framework por trás).

```
// npm install @nestjs/cqrs
import { CommandHandler, ICommandHandler, Command, CommandBus, EventBus } from
  ↪ '@nestjs/cqrs';
import { Injectable } from '@nestjs/common';

// COMMAND - representa uma INTENÇÃO de mudar estado (nome no imperativo)
class ConfirmarPedidoCommand {
  constructor(public readonly pedidoId: string) {}
}

// HANDLER - cada Command tem exatamente UM handler
@CommandHandler(ConfirmarPedidoCommand)
class ConfirmarPedidoHandler implements ICommandHandler<ConfirmarPedidoCommand>
  ↪ {
  constructor(private readonly eventBus: EventBus) {}

  async execute(command: ConfirmarPedidoCommand): Promise<void> {
    console.log(`Confirmando pedido ${command.pedidoId}`);
    // ... busca o agregado, chama pedido.confirmar(), salva no repositório ...
    this.eventBus.publish({ pedidoId: command.pedidoId, tipo:
      ↪ 'PedidoConfirmado' }); // dispara evento de domínio
  }
}

// CONTROLLER - só despacha o Command, não sabe COMO ele é processado
@Injectable()
class PedidosController {
  constructor(private readonly commandBus: CommandBus) {}

  async confirmar(pedidoId: string) {
    return this.commandBus.execute(new ConfirmarPedidoCommand(pedidoId));
  }
}
```

```
}
```

Dica de Pleno

CQRS **não exige** dois bancos de dados separados (um de escrita, outro de leitura) — essa é uma evolução opcional, mais avançada, geralmente chamada de “CQRS com modelos de leitura/escrita separados fisicamente”. A forma mais simples e mais comum de CQRS (a que o `@nestjs/cqrs` facilita) é só a separação **lógica** de código — Commands e Queries são classes diferentes, processadas por handlers diferentes, mesmo que ambos, por baixo, consultem o mesmo banco. Não confunda “praticar CQRS” com “obrigatoriamente ter Event Sourcing e dois bancos” — são conceitos relacionados, mas independentes.

Armadilha Comum

Adotar CQRS (e todo o cerimonial de Commands, Handlers, Events) para operações CRUD simples de um sistema pequeno é um caso clássico de over-engineering: um `CriarUsuarioCommand` + `CriarUsuarioHandler` para um cadastro que só faz um INSERT sem regra de negócio nenhuma adiciona indireção (mais arquivos, mais saltos para seguir o fluxo de código) sem nenhum ganho real. CQRS paga seu custo quando **leitura e escrita têm necessidades genuinamente diferentes** (leitura precisa de agregações complexas/desnormalizadas para performance, escrita precisa de forte consistência transacional) — não como padrão default para todo o sistema.

Cenário de Falha em Produção

Um time adota `@nestjs/cqrs` para 100% dos endpoints de um sistema administrativo interno de baixíssimo tráfego (dezenas de usuários), incluindo CRUDs triviais. O onboarding de novos desenvolvedores no time passa a levar semanas a mais só para entender o fluxo “Controller → Command → CommandBus → Handler → Repository” para uma operação que poderia ser um método direto de Service — a complexidade accidental introduzida pelo padrão supera, de longe, a complexidade de negócio real do sistema, e a velocidade de entrega do time cai mensuravelmente sem ganho de qualidade correspondente.

Dica de Produção

Antes de adotar CQRS via `@nestjs/cqrs` em um novo módulo, faça a pergunta inversa de “por que não usar”: se a resposta para “sua leitura e escrita desse domínio têm requisitos de performance ou de modelo de dados genuinamente diferentes?” é “não, são parecidos”, um Service direto (sem Command/Query/Handler) é a escolha mais simples e igualmente correta. Reserve CQRS explícito para os módulos onde a divergência entre leitura e escrita é real e mensurável — tipicamente, módulos com dashboards/relatórios complexos ao lado de escritas transacionais simples.

2.9 Testes no NestJS

Antes de chegar ao `TestingModule` do Nest, vale mostrar a camada mais barata e mais numerosa da pirâmide de testes (adiantada na Dica de Produção mais abaixo): testar **Entities e Value Objects puros**, sem nenhuma dependência do Nest, sem `TestingModule`, sem mock nenhum — porque, como visto nas seções 1.2 e 1.3, essas classes não têm dependência de framework, então não precisam de nenhuma infraestrutura de teste especial:

```
// npx vitest run dinheiro.spec.ts - ZERO import de @nestjs/*, ZERO mock
import { describe, it, expect } from 'vitest';

class Dinheiro {
  private readonly centavos: number;
  constructor(centavos: number, private readonly moeda: string) {
    if (centavos < 0) throw new Error('Dinheiro não pode ser negativo');
    this.centavos = centavos;
  }
  somar(outro: Dinheiro): Dinheiro {
    if (outro.moeda !== this.moeda) throw new Error('Moedas incompatíveis');
    return new Dinheiro(this.centavos + outro.centavos, this.moeda);
  }
  get valorEmCentavos() { return this.centavos; }
}

describe('Dinheiro (Value Object)', () => {
  it('soma dois valores da mesma moeda corretamente', () => {
    const total = new Dinheiro(1000, 'BRL').somar(new Dinheiro(500, 'BRL'));
    expect(total.valorEmCentavos).toBe(1500); // sem mock, sem setup - só a
    ↪ classe e o teste
  });

  it('rejeita construção com valor negativo', () => {
    expect(() => new Dinheiro(-100, 'BRL')).toThrow('Dinheiro não pode ser
    ↪ negativo');
  });

  it('rejeita somar moedas diferentes', () => {
    expect(() => new Dinheiro(100, 'BRL').somar(new Dinheiro(100,
    ↪ 'USD'))).toThrow('Moedas incompatíveis');
  });
});
```

Dica de Pleno

Repare que não existe nenhum `beforeEach` montando um `TestingModule`, nenhum provider, nenhum banco em memória — é só `new Dinheiro(...)` e uma asserção. Essa é a prova prática do argumento central do DDD: se a sua Entity ou Value Object exige qualquer infraestrutura de teste (mock de repositório, subir módulo do

Nest) só para testar uma regra de negócio simples como “não aceitar valor negativo”, isso é sinal de que uma dependência de framework vazou para dentro do domínio — o teste de um objeto de domínio puro deveria sempre ser tão direto quanto o exemplo acima.

O Nest oferece um `TestingModule` que replica o mesmo mecanismo de DI usado em produção, mas permite **sobrescrever providers por mocks** de forma cirúrgica — isso é o que possibilita testar um Controller ou Service isoladamente, sem subir a aplicação inteira nem depender de banco real.

```
// npm install -D @nestjs/testing vitest
// npx vitest run pedidos.service.spec.ts
import { Test } from '@nestjs/testing';
import { describe, it, expect, vi } from 'vitest';

class PedidoRepository {
  async buscarPorId(_id: string): Promise<any> { throw new Error('não
    ↳ implementado'); }
}

class PedidosService {
  constructor(private readonly repositorio: PedidoRepository) {}
  async buscar(id: string) {
    const pedido = await this.repositorio.buscarPorId(id);
    if (!pedido) throw new Error('Pedido não encontrado');
    return pedido;
  }
}

describe('PedidosService', () => {
  it('lança erro quando o pedido não existe', async () => {
    const moduloDeTeste = await Test.createTestingModule({
      providers: [
        PedidosService,
        { provide: PedidoRepository, useValue: { buscarPorId:
          ↳ vi.fn().mockResolvedValue(null) } }, // MOCK
      ],
    }).compile();

    const service = moduloDeTeste.get(PedidosService);

    await expect(service.buscar('id-inexistente')).rejects.toThrow('Pedido não
      ↳ encontrado');
  });
});
```

Dica de Pleno

Usar `Test.createTestingModule` (em vez de simplesmente instanciar `new PedidosService(mockRepo)` manualmente) parece redundante em exemplos simples como este, mas se paga em componentes com árvores de dependência mais profundas — o `TestingModule` resolve toda a cadeia de DI automaticamente, incluindo dependências de dependências, então você só precisa mockar explicitamente o que o teste realmente quer controlar, deixando o resto ser resolvido normalmente pelo container (ou também mockado, se preferir isolamento total).

Por Baixo dos Panos

Para testes de integração mais realistas (não unitários), o Nest também oferece `app.getHttpAdapter()` combinado com bibliotecas como `supertest`, permitindo subir a aplicação Nest completa **em memória** (sem bind numa porta TCP real) e disparar requisições HTTP de verdade contra ela. Isso testa a cadeia inteira — Guards, Pipes, Interceptors, Exception Filters (seção 2.3) — de forma muito mais próxima do comportamento real de produção do que um teste unitário isolado de um único provider, ao custo de ser mais lento e menos granular na hora de apontar exatamente onde algo quebrou.

Dica de Produção

A pirâmide de testes recomendada para um projeto Nest com DDD segue a mesma lógica de qualquer arquitetura em camadas: teste as **Entities e Value Objects do domínio** com testes unitários puros, sem `TestingModule` nenhum (são classes TypeScript simples, sem dependência do Nest) — são os testes mais rápidos e mais numerosos. Teste os **Use Cases/Application Services** com `TestingModule` e repositórios mockados (como no exemplo acima) — volume médio. Reserve testes de integração com `supertest` (subindo a aplicação completa) para os fluxos críticos de ponta a ponta — poucos, porque são os mais lentos e mais caros de manter.

Desafio de Mentoria

O time está debatendo a estratégia de testes para o módulo `PedidosModule` estruturado em camadas (seção 2.7). Um colega propõe: “vamos escrever testes de integração com `supertest` para tudo, é mais realista e cobre mais coisa de uma vez”. Como Advogado do Diabo, usando a pirâmide de testes descrita acima: em que cenário concreto essa estratégia “só testes de integração” vira um problema de velocidade de CI e de dificuldade de localizar a causa raiz de uma falha, à medida que o módulo cresce de 5 para 50 casos de uso? Que sinal concreto (tempo de execução da suíte, dificuldade em nomear o que quebrou) indicaria que é hora de reequilibrar a pirâmide para mais testes unitários de domínio?

Fim da Parte 2

Nesta parte, conectamos os conceitos táticos de DDD da Parte 1 ao NestJS: vimos os blocos fundamentais do framework (Módulos, Controllers, Providers — 2.1), os mecanismos

de “cola” (DI e Escopos — 2.2; Pipes/Guards/Interceptors — 2.3; Decorators customizados — 2.5), a dupla DTO+Mapper que protege o domínio na fronteira (2.4 e 2.6), a estrutura de pastas que materializa as 4 camadas de DDD dentro de um módulo Nest (2.7), e dois tópicos avançados — CQRS (2.8) e a pirâmide de testes (2.9).

Com a estrutura de código definida — um domínio isolado, uma camada de aplicação que orquestra, e uma interface HTTP na borda —, o próximo passo é decidir se ela será implantada como um único bloco (o `PedidosModule` vivendo dentro do mesmo processo que `CientesModule`, `EstoqueModule` etc.) ou como serviços independentes, cada um com seu próprio deploy, seu próprio banco, e comunicando-se pela rede. É essa decisão — quando, como e a que custo — que a Parte 3 explora.

Chapter 3

Parte 3 — Do Monólito aos Microserviços

3.1 O que são microserviços e quando NÃO usar

Microserviços são um estilo arquitetural onde um sistema é decomposto em **serviços independentes, implantados separadamente**, cada um dono do seu próprio banco de dados, comunicando-se pela rede (não por chamada de função in-process). A motivação real não é “microserviços são melhores que monólito” — é **escalar times de forma independente**: cada serviço pode ser desenvolvido, testado e implantado por uma equipe pequena, sem coordenar deploy com outras 10 equipes.

Dica de Pleno

A pergunta mais importante antes de adotar microserviços não é técnica, é organizacional — conhecida como **Lei de Conway** (“sistemas espelham a estrutura de comunicação da organização que os constrói”). Se você tem um time de 5 pessoas, microserviços tendem a multiplicar o trabalho operacional (deploy, monitoramento, versionamento de contrato entre 8 serviços) sem o benefício real de paralelização de times, porque não existem times separados para paralelizar. Martin Fowler resume isso como o “**MonolithFirst**”: comece com um monólito bem modularizado (usando os limites de Bounded Context do DDD, seção 1.7, como costura interna) e só extraia serviços quando um limite organizacional ou de escala real justificar o custo.

Por Baixo dos Panos

A complexidade que “desaparece” ao sair de um monólito na verdade **não desaparece — ela migra**. Uma chamada de função (`pedidosService.criar(dados)`, que sempre funciona ou lança exceção de forma síncrona e local) vira uma chamada de rede (HTTP, gRPC ou mensageria) que pode falhar de formas que uma chamada local nunca falha: timeout, partição de rede, serviço temporariamente indisponível, resposta parcial. Esse conjunto de falhas tem nome — as **Falácias da Computação Distribuída** (“a rede é confiável”, “a latência é zero”, “a largura de banda

é infinita” — todas falsas premissas que times habituados a monólito assumem sem perceber ao migrar para microsserviços) — e ignorá-las é a causa raiz da maioria dos incidentes graves em arquiteturas distribuídas recém-migradas.

Cenário de Falha em Produção

Uma startup com 6 desenvolvedores decide, por influência de blog posts de empresas como Netflix e Uber, quebrar seu monólito em 12 microsserviços “para escalar”. Seis meses depois, uma feature simples (“adicionar campo de observação ao pedido”) que levaria 2 horas no monólito antigo agora exige coordenar mudanças em 4 serviços diferentes (Pedidos, Faturamento, Notificação, Relatórios), cada um com seu próprio pipeline de deploy, e uma mudança de contrato mal comunicada entre dois desses serviços (o campo novo não foi propagado corretamente no evento assíncrono que um consome do outro) causa inconsistência de dados silenciosa detectada só 3 semanas depois, na reconciliação mensal — um tipo de bug que simplesmente não existiria em um monólito com uma única transação de banco.

Dica de Produção

Um sinal maduro para saber se sua organização está pronta (ou precisa) de microsserviços: você já tem múltiplos times trabalhando no mesmo monólito e **frequentemente** bloqueados uns pelos outros — esperando review, esperando janela de deploy, evitando mexer em código “de outro time” por medo de quebrar algo não relacionado? Se sim, o custo organizacional do monólito já superou o custo operacional de distribuir. Se sua dor real é “performance” ou “escala técnica” (não coordenação de times), considere primeiro otimizações dentro do monólito (cache, réplicas de leitura, filas internas) — que resolvem boa parte dos problemas de escala técnica sem o custo de uma arquitetura distribuída completa.

Da decisão à prática: migração pelo padrão Strangler Fig

Depois de decidir extrair um serviço, a pergunta seguinte é **como**, sem parar o time por meses nem arriscar um “big bang” de alto risco. A resposta mais usada na indústria é o padrão **Strangler Fig** (nome de Martin Fowler, inspirado na planta que cresce ao redor de uma árvore hospedeira até substituí-la por completo): em vez de reescrever tudo de uma vez, você coloca um proxy/gateway na frente do monólito e extrai **um Bounded Context por vez** — cada extração implementa aquele contexto como um serviço novo, e o proxy passa a rotear só o tráfego daquele contexto específico para o serviço novo, enquanto todo o resto continua batendo no monólito, inalterado. Repita esse processo, contexto por contexto, até o monólito “encolher” até sobrar pouco ou nada dele — daí o nome.

Dica de Pleno

É exatamente por isso que os limites de **Bounded Context** (seção 1.7) importam tanto antes de cogitar microsserviços — eles não são só uma ferramenta de mode-

lagem de domínio, são o **mapa de onde cortar** quando a extração via Strangler Fig começa. Um sistema sem Bounded Contexts claros não tem “costuras” naturais para extrair um serviço por vez; a extração vira, na prática, um big bang disfarçado, porque não existe um pedaço isolado o suficiente para tirar sozinho sem arrastar o resto. É também por isso que a Armadilha Comum da seção 2.7 (vazamento de framework para dentro do `domain/`) é tão cara: cada acoplamento não-planejado é mais um ponto de fricção no dia em que esse Bounded Context precisar virar um serviço próprio.

3.2 Comunicação síncrona vs assíncrona

Antes de qualquer padrão de eventos entre serviços, a decisão mais básica é: essa comunicação deveria bloquear quem chama, esperando resposta, ou não?

	Síncrona (REST/gRPC)	Assíncrona (mensageria)
Como funciona	Serviço A chama Serviço B e espera a resposta	Serviço A publica um evento/mensagem e segue em frente, sem esperar
Acoplamento temporal	Alto — se B está fora do ar, A falha (ou fica bloqueado)	Baixo — B pode estar fora do ar; a mensagem fica na fila até B voltar
Consistência	Mais fácil de raciocinar (resposta imediata, sucesso/erro claro)	Consistência eventual — exige lidar com “ainda não processado”
Exemplo de uso	“Buscar detalhes do cliente para exibir na tela agora”	“Notificar que um pedido foi confirmado, para quem quiser reagir”

Dica de Pleno

A heurística mais prática para decidir entre os dois: se o chamador **precisa** do resultado imediatamente para continuar seu próprio fluxo (ex: “validar se o cartão de crédito é válido antes de finalizar a compra”), use comunicação síncrona. Se o chamador só precisa **informar que algo aconteceu**, sem depender de uma resposta imediata para prosseguir (ex: “informar o time de logística que pode preparar o envio”), use mensageria assíncrona — isso desacopla os serviços no tempo, e um pico de tráfego ou indisponibilidade temporária do consumidor não derruba o produtor.

Cenário de Falha em Produção

Um serviço de Checkout chama, de forma **síncrona**, três outros serviços em sequência antes de confirmar um pedido: Estoque, Frete e Notificação — os três via chamada HTTP direta, aguardando resposta de cada um antes de seguir para o próximo. O serviço de Notificação (o menos crítico dos três, já que só dispara

um e-mail) começa a apresentar lentidão em um deploy problemático, e como o Checkout espera sua resposta de forma síncrona **antes de confirmar o pedido para o cliente**, a lentidão do serviço menos importante do fluxo (notificação por e-mail) derruba a taxa de conversão de vendas do site inteiro — um caso clássico de **acoplamento temporal desnecessário**, onde um serviço não-crítico para o resultado principal (a venda em si) foi colocado no caminho crítico síncrono.

Dica de Produção

Ao desenhar (ou revisar) um fluxo entre serviços, trace explicitamente: “se o Serviço X estivesse fora do ar agora, o fluxo principal ainda deveria funcionar?”. Para qualquer serviço onde a resposta é “sim, o fluxo principal não deveria depender disso”, mova essa chamada de síncrona para assíncrona (publicar um evento e seguir em frente) — isso isola falhas de serviços secundários do caminho crítico de negócio. Em produção, dashboards que correlacionam “latência/erro do Serviço X” com “queda de conversão/taxa de sucesso do fluxo principal” são o jeito mais rápido de identificar esses acoplamentos temporais desnecessários depois do fato — antes do fato, a pergunta acima em design review é bem mais barata.

3.3 NestJS Microservices: @MessagePattern vs @EventPattern

O Nest tem suporte nativo a microsserviços através do pacote `@nestjs/microservices`, que abstrai o **transporte** (TCP, Redis, RabbitMQ, Kafka, gRPC, NATS) atrás da mesma API de decorators (`@MessagePattern`, `@EventPattern`) usada em controllers HTTP — trocar de transporte é, na maior parte dos casos, uma mudança de configuração, não de código de negócio. Esses dois decorators materializam, dentro do Nest, exatamente a distinção conceitual da seção 3.2:

	@MessagePattern	@EventPattern
Semântica	Request/response — o chamador espera retorno	Fire-and-forget — sem retorno esperado
Análogo conceitual	Chamada síncrona (REST)	Mensageria assíncrona
Acoplamento temporal	O chamador fica bloqueado até a resposta (ou timeout)	Nenhum — o chamador segue em frente
Quando usar	Precisa do dado de volta para decidir o próximo passo	Só precisa informar/reagir a um fato que já aconteceu

```
// npm install @nestjs/microservices amqplib amqp-connection-manager
import { Controller } from '@nestjs/common';
import { MessagePattern, EventPattern, Payload, Transport } from
  ↳ '@nestjs/microservices';
import { NestFactory } from '@nestjs/core';
```

```
// CONTROLLER de microserviço - mesma sintaxe de decorators do HTTP, transporte
↳ diferente
@Controller()
class PedidosMicroserviceController {
  // MessagePattern = request/response (o chamador ESPERA uma resposta) -
  ↳ análogo a REST síncrono
  @MessagePattern('pedidos.buscar_por_id')
  buscarPorId(@Payload() dados: { id: string }) {
    return { id: dados.id, status: 'CONFIRMADO' }; // o chamador RECEBE isso de
    ↳ volta
  }

  // EventPattern = fire-and-forget (o chamador NÃO espera resposta) - análogo a
  ↳ mensageria assíncrona
  @EventPattern('pedido.confirmado')
  aoConfirmarPedido(@Payload() dados: { pedidoId: string }) {
    console.log(`[EVENTO] Reagindo à confirmação do pedido ${dados.pedidoId}`);
    // não existe "retorno" aqui - quem publicou o evento já seguiu em frente
  }
}

// bootstrap do microserviço (main.ts) - RabbitMQ como transporte, por exemplo
// const app = await NestFactory.createMicroservice(AppModule, {
//   transport: Transport.RMQ,
//   options: { urls: ['amqp://localhost:5672'], queue: 'pedidos_queue' },
// });
// await app.listen();
```

Dica de Pleno

Escolher o decorator errado tem a mesma consequência prática da seção 3.2 — usar `@MessagePattern` para algo que deveria ser fire-and-forget acopla temporalmente o chamador ao tempo de resposta do serviço chamado, mesmo estando “tecnicamente” sobre um transporte de mensageria como RabbitMQ.

Armadilha Comum

Um erro comum de quem está começando com `@nestjs/microservices` é assumir que `@MessagePattern` sobre RabbitMQ tem as mesmas garantias de reentrega e persistência que `@EventPattern`. Na prática, o Nest implementa `@MessagePattern` sobre RabbitMQ usando um padrão de fila de resposta temporária (RPC sobre mensageria), que **não** foi desenhado para ser tão resiliente a longas indisponibilidades do consumidor quanto o padrão pub/sub de `@EventPattern`. Para garantias fortes de entrega mesmo com o consumidor fora do ar por horas, prefira `@EventPattern` (ou uma fila de trabalho dedicada, com retry e dead-letter queue configurados explicitamente), não `@MessagePattern`.

Cenário de Falha em Produção

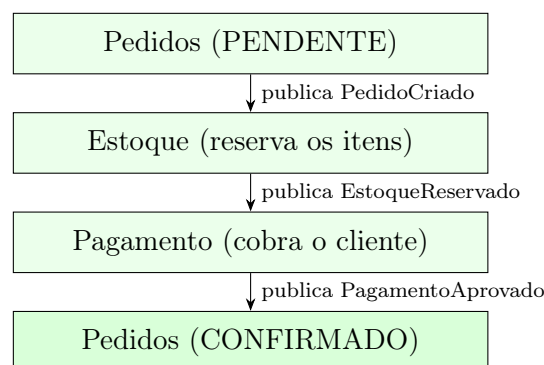
Um serviço de Checkout usa `@MessagePattern` (RPC síncrono sobre RabbitMQ) para consultar o serviço de Estoque em cada finalização de compra, com um timeout padrão relativamente curto configurado no cliente. Em um pico de Black Friday, o serviço de Estoque fica temporariamente sobrecarregado e começa a responder mais devagar que o timeout configurado — o Checkout começa a lançar exceções de timeout em cascata, e como não havia circuit breaker (seção 3.6) nem fallback configurado, **cada** requisição de checkout tenta a chamada, espera o timeout completo, e falha — consumindo recursos (conexões, memória de requisições pendentes) do próprio serviço de Checkout enquanto espera, agravando ainda mais a situação em vez de “falhar rápido” e liberar recursos.

Dica de Produção

Ao configurar qualquer chamada de `@MessagePattern` (RPC síncrono) entre microsserviços, trate `timeout` explícito como obrigatório, não opcional — o padrão de alguns clientes é esperar indefinidamente, o que em um cenário de degradação em cascata é o pior comportamento possível. Combine isso com métricas específicas de “tempo de resposta de chamadas RPC entre serviços” (não só HTTP externo) no seu painel de observabilidade — em uma arquitetura de microsserviços, boa parte da latência percebida pelo usuário final é soma de latências de chamadas internas entre serviços, invisíveis se você só monitora a borda (o endpoint público).

3.4 Consistência eventual: o padrão Saga

Em um monólito, uma operação com múltiplas etapas (“debitar saldo, reservar estoque, agendar entrega”) roda dentro de **uma única transação de banco** — se qualquer etapa falha, tudo é revertido automaticamente. Quando essas etapas vivem em serviços diferentes, cada um com seu próprio banco, **não existe transação distribuída simples** — e é aqui que entra o padrão **Saga**: uma sequência de transações locais, onde cada etapa publica um evento que dispara a próxima, e **cada etapa também define uma ação de compensação** para desfazer seu efeito caso uma etapa posterior falhe.



Fluxo feliz



Fluxo de compensação (pagamento falha)

Dica de Pleno

A existência da ação de **compensação** (liberar a reserva de estoque quando o pagamento é recusado) é o que diferencia uma Saga de simplesmente “publicar eventos e torcer para dar certo”. Toda etapa de uma Saga que **muda estado** precisa, desde o desenho inicial, de uma contrapartida explícita que desfça esse efeito — não é algo que se adiciona depois “se algo der errado”; é parte do design da etapa desde o início, exatamente como um ROLLBACK é parte do design de uma transação de banco.

Traduzindo o diagrama acima para um fluxo coreografado passo a passo:

```

// Ilustração conceitual de uma Saga coreografada (cada serviço reage a eventos,
↪ sem orquestrador central)

// 1. Serviço de Pedidos: cria pedido em status PENDENTE, publica evento
//    -> PedidoCriado { pedidoId, itens }

// 2. Serviço de Estoque: reage a PedidoCriado, tenta reservar itens
//    SUCESSO -> publica EstoqueReservado { pedidoId }
//    FALHA   -> publica EstoqueIndisponivel { pedidoId } (ação de COMPENSAÇÃO
↪   adiante)

// 3. Serviço de Pagamento: reage a EstoqueReservado, tenta cobrar
//    SUCESSO -> publica PagamentoAprovado { pedidoId }
//    FALHA   -> publica PagamentoRecusado { pedidoId }

// 4a. Serviço de Pedidos: reage a PagamentoAprovado -> status CONFIRMADO (fluxo
↪   feliz)

// 4b. Serviço de Estoque: reage a PagamentoRecusado -> LIBERA a reserva
↪   (COMPENSAÇÃO da etapa 2)
  
```

```
// 4c. Serviço de Pedidos: reage a EstoqueIndisponivel OU PagamentoRecusado ->  
→ status CANCELADO
```

Por baixo dos panos: Saga orquestrada vs coreografada

Existem duas formas de implementar Saga:

- **Coreografada** (a do diagrama acima): cada serviço reage a eventos de forma descentralizada, sem um “maestro” central. Mais simples para poucos passos, mas fica difícil de visualizar/depurar à medida que o número de serviços cresce — o fluxo completo não está escrito em lugar nenhum, só “emerge” da soma dos handlers espalhados pelos serviços.
- **Orquestrada**: um serviço central (o Orquestrador da Saga) chama explicitamente cada etapa em sequência e decide quando compensar. Centraliza a lógica do fluxo em um lugar só (mais fácil de entender e depurar), ao custo de criar um serviço com conhecimento sobre todos os outros — um acoplamento a mais, porém concentrado e visível, em vez de espalhado e implícito.

Cenário de Falha em Produção

Uma Saga coreografada de checkout tem sua etapa de compensação de estoque (`EstoqueIndisponivel` → liberar reserva) implementada, mas um deploy no serviço de Pagamento introduz um bug que faz `PagamentoRecusado` ser publicado com o `pedidoId` **vazio** em alguns casos de borda (erro de serialização). O serviço de Estoque, que espera reagir a esse evento para liberar a reserva, recebe o evento malformado, falha silenciosamente ao tentar localizar o pedido pelo ID vazio, e **nunca libera a reserva** — itens ficam “presos” como reservados indefinidamente, para pedidos que na verdade foram cancelados, causando uma divergência crescente entre estoque físico e estoque reservado, percebida só quando a operação nota falta de itens que deveriam estar disponíveis.

Dica de Produção

Toda Saga em produção precisa de uma forma de **auditoria e recuperação manual** para os casos em que uma etapa de compensação falha silenciosamente. Um “livro-razão” de eventos da Saga (quem publicou o quê, quando, e se foi processado com sucesso) é o que permite a um humano (ou um job de reconciliação automatizado) identificar e corrigir Sagas travadas no meio do caminho — sem essa auditoria, o único sinal costuma ser um sintoma indireto e tardio (como “estoque reservado maior que estoque físico”), muito mais difícil de rastrear até a causa raiz do que consultar diretamente o histórico de eventos daquela Saga específica.

Desafio de Mentoria

Você está desenhando o fluxo de cancelamento de assinatura de um SaaS, que envolve 4 serviços (Assinaturas, Cobrança, Acesso/Permissões, Notificação). Um colega sugere Saga coreografada “porque é mais simples de começar”; outro sugere Saga orquestrada “porque vai ficar mais fácil de depurar quando o fluxo crescer”. Como Advogado do Diabo: dado que hoje são 4 serviços mas o roadmap prevê integração com um 5º (Parceiros, para revenda), em que ponto a Saga coreografada realmente começa a doer o suficiente para justificar migrar para orquestrada — e essa migração pode ser feita incrementalmente, ou exige reescrever o fluxo do zero?

3.5 Quando usar eventos assíncronos — e quando NÃO usar

Eventos e Sagas resolvem um problema real (desacoplamento temporal, resiliência a indisponibilidade parcial), mas custam **complexidade genuína**: consistência eventual (o sistema pode estar temporariamente “errado” entre uma etapa e outra), depuração mais difícil (o fluxo não é uma stack trace linear, é uma sequência de mensagens espalhadas em logs de serviços diferentes), e a necessidade de projetar compensação para toda escrita.

Dica de Pleno

Não adote eventos assíncronos “porque é a arquitetura moderna” — adote quando o custo de acoplamento temporal síncrono (seção 3.2) já é um problema real e mensurável (timeouts em cascata, indisponibilidade de um serviço secundário derrubando o fluxo principal). Para dois serviços que sempre estarão no ar juntos, com baixa latência entre eles, e onde o chamador realmente precisa do resultado imediato, uma chamada síncrona simples é mais fácil de entender, depurar e operar do que uma Saga — não troque simplicidade por sofisticação sem um problema concreto para resolver.

Essa mesma disciplina de “não adotar o padrão sofisticado sem um problema concreto” já apareceu antes neste guia — na decisão entre Nest e Express puro (seção 2.1) e na decisão de adotar CQRS (seção 2.8). O fio condutor é sempre o mesmo: cada padrão tem custo real, e a maturidade está em medir esse custo contra o problema que você de fato tem, não em aplicar a ferramenta mais sofisticada disponível.

3.6 API Gateway, BFF e Circuit Breaker

Três padrões que resolvem problemas distintos, mas frequentemente aparecem juntos na borda de uma arquitetura de microsserviços:

- **API Gateway**: um único ponto de entrada que roteia requisições externas para o microsserviço correto, geralmente também cuidando de autenticação, rate limiting

e agregação de respostas de múltiplos serviços — evita que o cliente (app mobile, frontend web) precise conhecer os endereços internos de cada um dos N serviços.

- **BFF (Backend for Frontend)**: uma variação do API Gateway, mas **um gateway dedicado por tipo de cliente** (um BFF para o app mobile, outro para o painel web administrativo) — porque as necessidades de dados de cada frontend costumam divergir (o mobile quer uma resposta enxuta; o painel admin quer dados ricos e detalhados), e um gateway único genérico tende a virar um meio-termo ruim para todos.
- **Circuit Breaker**: um padrão de resiliência que **para de tentar chamar** um serviço que está falhando repetidamente, por um período, retornando uma falha rápida (ou um fallback) em vez de continuar tentando e acumulando timeouts — exatamente o problema descrito no cenário de falha da seção 3.3.

```
// npm install opossum
import CircuitBreaker from 'opossum';

async function chamarServicoEstoque(produtoId: string): Promise<{ disponivel:
  ↳ boolean }> {
  const resposta = await fetch(`http://servico-estoque/produtos/${produtoId}`);
  if (!resposta.ok) throw new Error('Serviço de estoque indisponível');
  return resposta.json();
}

const opcoes = {
  timeout: 3000, // se não responder em 3s, conta como falha
  errorThresholdPercentage: 50, // se 50% das últimas chamadas falharem...
  resetTimeout: 10000, // ...abre o circuito por 10s antes de tentar de
  ↳ novo
};

const breaker = new CircuitBreaker(chamarServicoEstoque, opcoes);

// FALLBACK - o que responder quando o circuito está ABERTO (serviço considerado
↳ indisponível)
breaker.fallback(() => ({ disponivel: false })); // assume indisponível, não
↳ trava o fluxo principal

breaker.on('open', () => console.log('[CIRCUIT BREAKER] Circuito ABERTO -
↳ parando de chamar o serviço de estoque'));
breaker.on('close', () => console.log('[CIRCUIT BREAKER] Circuito FECHADO -
↳ voltando ao normal'));

// uso: breaker.fire(produtoId) em vez de chamar chamarServicoEstoque
↳ diretamente
```

Dica de Pleno

O nome “circuit breaker” (disjuntor) é uma analogia elétrica direta e precisa: assim como um disjuntor físico “abre” o circuito para proteger a instalação elétrica de uma sobrecarga, o Circuit Breaker de software “abre” (para de tentar chamar o serviço com problema) para proteger o **serviço chamador** de gastar seus próprios recursos (conexões, threads, memória) esperando por um serviço que provavelmente vai continuar falhando pelos próximos segundos. Ele existe para proteger quem chama, não quem é chamado — embora, como efeito colateral, também dê um “respiro” ao serviço com problema, por reduzir a carga de chamadas recebidas enquanto o circuito está aberto.

Cenário de Falha em Produção

Um sistema com 3 BFFs (mobile, web, parceiros) cada um chamando os mesmos 5 microsserviços internos, mas sem nenhum Circuit Breaker configurado. Quando o serviço de Recomendações fica degradado (lento, não fora do ar completamente), **todos os 3 BFFs** continuam tentando chamá-lo normalmente, cada requisição esperando o timeout completo antes de desistir — o efeito cascata consome recursos nos 3 BFFs simultaneamente, e como eles compartilham a mesma infraestrutura de rede/proxy reverso na borda, a degradação de um único serviço interno de baixa criticidade (recomendações) acaba impactando a disponibilidade de fluxos completamente não relacionados (como login, que passa pelo mesmo proxy reverso saturado por conexões pendentes de outras chamadas).

Dica de Produção

Todo serviço interno chamado de forma síncrona por múltiplos consumidores deveria ter, no consumidor, Circuit Breaker configurado com um **fallback** explícito e **pensado para o caso de negócio específico** — nem sempre “retornar vazio” é o fallback certo (para recomendações, uma lista vazia ou genérica é aceitável; para verificação de saldo antes de um débito, um fallback “assumir saldo suficiente” seria perigoso e incorreto). Ao configurar cada Circuit Breaker, force explicitamente a pergunta “o que é seguro fazer quando esse serviço específico está indisponível?” — a resposta muda completamente dependendo da criticidade e da natureza da informação.

3.7 Observabilidade distribuída

Em um monólito, uma stack trace de erro aponta diretamente para a linha de código problemática. Em microsserviços, um erro percebido pelo usuário pode ter se originado em qualquer um dos N serviços que participaram daquela requisição — e sem instrumentação específica, descobrir **qual** serviço causou a lentidão ou o erro vira um exercício de tentativa e erro. A ferramenta central para resolver isso é o **Correlation ID** (ou Trace ID): um identificador único gerado na borda (API Gateway) e propagado, via header HTTP, por **todas** as chamadas subsequentes entre serviços, permitindo reconstruir a jornada completa de uma requisição em ferramentas de tracing distribuído (Jaeger, Zipkin, ou

soluções gerenciadas como Datadog APM).

```
// npm install @nestjs/common uuid
import { Injectable, NestMiddleware } from '@nestjs/common';
import { randomUUID } from 'crypto';

// MIDDLEWARE na borda (API Gateway) - gera ou propaga o Correlation ID
@Injectable()
class CorrelationIdMiddleware implements NestMiddleware {
  use(req: any, res: any, next: () => void) {
    // reaproveita o ID se já veio de outro serviço; gera um novo se essa é a
    // ↳ entrada do sistema
    req.correlationId = req.headers['x-correlation-id'] || randomUUID();
    res.setHeader('x-correlation-id', req.correlationId);
    next();
  }
}

// toda chamada SAINTE para outro serviço deve PROPAGAR o mesmo header
async function chamarServicoEstoque(correlationId: string, produtoId: string) {
  return fetch(`http://servico-estoque/produtos/${produtoId}`, {
    headers: { 'x-correlation-id': correlationId }, // propaga - sem isso, o
    // ↳ trace "quebra" nesse ponto
  });
}
```

Dica de Pleno

O Correlation ID só é útil se **todo** log emitido durante o processamento daquela requisição — em **todos** os serviços que ela atravessa — incluir esse mesmo ID como campo estruturado (não só em texto livre). Isso é o que permite, em produção, uma busca simples como `correlationId:"abc-123"` no seu sistema de logs centralizado (Datadog, ELK) trazer **toda** a jornada da requisição, em ordem, através de todos os serviços — transformando um problema de “qual dos 6 serviços causou isso?” em uma busca de segundos, em vez de uma investigação manual serviço por serviço.

Por Baixo dos Panos

Tracing distribuído completo (não só Correlation ID para correlacionar logs, mas visualização de **spans** — cada etapa de processamento, com início/fim e duração, aninhada em uma árvore) segue o padrão **OpenTelemetry** (padrão aberto, sucessor unificado do OpenTracing e OpenCensus), hoje suportado nativamente por praticamente todas as ferramentas de observabilidade comerciais e open-source. Um “trace” é a jornada completa de uma requisição; cada “span” dentro dele representa uma unidade de trabalho (uma chamada HTTP a outro serviço, uma query de banco) — visualizar a árvore de spans de uma requisição lenta mostra imediatamente **qual span específico** consumiu a maior parte do tempo total, sem precisar adivinhar.

Cenário de Falha em Produção

Um sistema com 8 microsserviços tem Correlation ID implementado em 7 deles, mas um serviço mais antigo (escrito antes da convenção ser adotada pelo time) não propaga o header para as chamadas que ele mesmo faz a outros serviços — sempre que uma requisição passa por esse serviço específico, o trace “quebra” ali, e os logs dos serviços chamados **depois** dele aparecem com um Correlation ID novo e desconectado do original. Durante um incidente de produção de lentidão intermitente, o time gasta horas tentando correlacionar manualmente logs de timestamps próximos entre dois grupos de serviços que, na verdade, faziam parte da mesma requisição — até alguém notar, pela ausência do header numa chamada específica, qual serviço era o elo quebrado da cadeia de propagação.

Dica de Produção

Propagação de Correlation ID (e de contexto de tracing em geral) é o tipo de convenção que **não pode** depender de cada desenvolvedor lembrar de implementar manualmente em cada nova chamada HTTP entre serviços — o risco de esquecimento é alto e o custo de um “elo quebrado” na cadeia é caro de diagnosticar, como no cenário acima. A forma robusta de garantir isso é centralizar a propagação em uma camada compartilhada (um HTTP client wrapper usado por **todos** os serviços, ou instrumentação automática via OpenTelemetry SDK, que intercepta chamadas HTTP/gRPC salientes e injeta o header de trace automaticamente, sem exigir código explícito em cada chamada individual).

Na prática hoje: OpenTelemetry com auto-instrumentação

O `CorrelationIdMiddleware` da seção anterior é didático para entender o **conceito**, mas na prática ninguém deveria escrever essa propagação manualmente hoje em dia — o SDK de auto-instrumentação do OpenTelemetry intercepta chamadas HTTP, gRPC e a maioria dos clientes populares (Prisma, `fetch`, `amqplib`) automaticamente, sem exigir nenhum middleware manual de propagação:

```
// tracing.ts - carregado ANTES de qualquer outro import da aplicação (node -r
↪ ./tracing.js main.js)
// npm install @opentelemetry/sdk-node @opentelemetry/auto-instrumentations-node
↪ @opentelemetry/exporter-trace-otlp-http
import { NodeSDK } from '@opentelemetry/sdk-node';
import { getNodeAutoInstrumentations } from
↪ '@opentelemetry/auto-instrumentations-node';
import { OTLPTraceExporter } from '@opentelemetry/exporter-trace-otlp-http';

const sdk = new NodeSDK({
  serviceName: 'servico-pedidos', // aparece como o "nome do nó" no grafo de
↪ tracing
  traceExporter: new OTLPTraceExporter({ url:
↪ 'http://otel-collector:4318/v1/traces' }),
```

```
instrumentations: [getNodeAutoInstrumentations()], // instrumenta HTTP, gRPC,  
  ↪ Prisma, etc. automaticamente  
});  
  
sdk.start();
```

Dica de Pleno

Repare que esse arquivo `tracing.ts` **não importa nada** do seu código de negócio — ele é carregado antes de tudo (`node -r ./tracing.js`) e faz *monkey-patching* das bibliotecas HTTP/banco do processo inteiro antes delas serem usadas. É esse truque de carregamento que permite instrumentação **sem** tocar em nenhuma rota, controller ou service da aplicação — o contraste com o `CorrelationIdMiddleware` manual da seção anterior é exatamente esse: o conceito (propagar um identificador através de toda a cadeia de chamadas) é o mesmo, mas a implementação real de produção substitui código explícito por configuração declarativa, cobrindo automaticamente até chamadas que você esqueceria de instrumentar manualmente.

Desafio de Mentoria

Sua equipe herdou um sistema de 3 anos, um monólito com controllers Express que fazem query SQL direto (`db.query('SELECT ...')`) misturada com validação e regra de negócio no mesmo arquivo — nenhuma das 4 camadas da seção 1.8 existe hoje. O tech lead pede um plano de “migração gradual para DDD, depois microsserviços” que não pare o desenvolvimento de features novas por meses. Como Advogado do Diabo: você reescreveria tudo de uma vez (big bang) ou extrairia camada por camada, endpoint por endpoint (strangler fig)? Que critério você usaria para escolher **qual** dos 40 endpoints do sistema migrar primeiro — o mais simples (para “aprender fazendo” com baixo risco) ou o que concentra mais bugs de regra de negócio duplicada (onde o ganho de ter um domínio único e testável é maior)? Justifique considerando que o time tem 4 desenvolvedores e não pode parar de entregar — e considere se essa migração deveria, desde já, mirar em Bounded Contexts que um dia virem microsserviços, ou se essa preocupação é prematura para o tamanho atual do time.

Chapter 4

Conclusão

DDD, NestJS e microsserviços resolvem três problemas em camadas diferentes, e é importante não confundi-los: **DDD** (Parte 1) é sobre como **modelar** a complexidade do negócio, independente de qualquer framework ou de o sistema ser distribuído ou não — Entity e Value Object são o vocabulário básico de modelagem; Aggregate e Aggregate Root definem a fronteira de consistência transacional; Domain Events permitem que essa fronteira avise o resto do sistema sobre o que aconteceu, sem acoplamento direto. **NestJS** (Parte 2) é uma escolha de **framework** que estrutura Injeção de Dependência, ciclo de vida de requisição e organização de módulos, e que acomoda bem os conceitos de DDD por convenção — DTO e Mapper protegem esse domínio inteiro do vocabulário e das constraints de qualquer dado que venha de fora, seja um cliente HTTP, seja um sistema legado. **Microsserviços** (Parte 3) é uma decisão de **arquitetura de implantação**, sobre se e como distribuir esse sistema em múltiplos processos — decisão que deveria vir **depois**, e não antes, de o domínio estar bem modelado e os limites de Bounded Context estarem claros; quando essa distribuição acontece, EDA e Saga levam a mesma ideia de Domain Events para além de um único processo, coordenando consistência entre serviços.

Sugestão de uso deste material:

1. Antes de escrever qualquer Mapper ou DTO (Parte 2), garanta que a distinção Entity/Value Object (1.2) e Aggregate/Aggregate Root (1.3) estão sólidas — são o vocabulário que todo o resto pressupõe. Rode os exemplos de código dessas seções sem framework nenhum primeiro.
2. Para as seções de NestJS (2.1 a 2.5), monte um projeto pequeno de verdade (`npx @nestjs/cli new`) e reproduza os exemplos rodando, não apenas lendo — o comportamento do pipeline de Guards/Interceptors/Pipes só “gruda” quando você vê a ordem de execução acontecer com `console.log`. Rode o exemplo da seção 2.7 mentalmente contra um Use Case real do seu projeto atual: consegue apontar onde estão o DTO, o Mapper, a Aggregate Root e a publicação de eventos?
3. Antes de adotar Sagas (3.4) para um novo fluxo, releia a seção 3.5 — a maior parte da complexidade accidental em sistemas distribuídos vem de eventos assíncronos adotados sem um problema concreto de acoplamento temporal para resolver.

4. Antes de estudar microsserviços (Parte 3) como uma meta a perseguir, releia a seção 3.1 — a maior parte das decisões ruins de arquitetura distribuída vem de pular essa reflexão organizacional antes da técnica.
5. Use a Tabela Consolidada de Referências (Apêndice A) como trilha de aprofundamento contínuo — os artigos de Martin Fowler linkados, em particular, são referências que a comunidade cita há mais de uma década por um motivo: envelheceram bem.

Dica de Pleno

A pergunta que separa Pleno de Sênior aqui não é “você sabe implementar um Aggregate, um Guard, ou uma Saga?” — é “você sabe reconhecer quando **não** usar cada um desses padrões, porque o problema que eles resolvem não é o problema que você tem agora?”. DDD, CQRS, Sagas e a própria decisão de ir para microsserviços são ferramentas com custo real; a maturidade está em saber medir esse custo contra o problema concreto, não em aplicar o padrão mais sofisticado disponível. E, complementarmente: se você tivesse que escolher uma única pergunta para auditar a saúde de DDD em um projeto existente, seria “meu domínio (Entities, Value Objects, Aggregates) importa alguma coisa de fora dele — um DTO, um driver de banco, um decorator de framework?”. Se a resposta é não, em qualquer lugar do código, você tem um domínio genuinamente protegido — e é exatamente isso que Mappers, Repositories e a disciplina de Aggregate Root, juntos, existem para garantir.

Appendix A

Referências e Leitura Adicional

Tópico	Links recomendados
DDD — Fundamentos	Eric Evans — DDD Reference
	Martin Fowler — Anemic Domain Model
DDD — Entities e Value Objects	Martin Fowler — Value Object
DDD — Agregados	Martin Fowler — DDD Aggregate
	Vaughn Vernon — Effective Aggregate Design
DDD — Bounded Context e Mappers/ACL	Martin Fowler — Bounded Context
	Microsoft — DDD-Oriented Microservice: the Domain Model Layer
Domain Events	Martin Fowler — Domain Event
NestJS — Fundamentos	NestJS Docs — First Steps
	NestJS Docs — Injection Scopes
	NestJS Docs — Custom Providers
NestJS — Pipeline de requisição	NestJS Docs — Guards
	NestJS Docs — Interceptors
	NestJS Docs — Pipes
	NestJS Docs — Exception Filters
NestJS — DTOs e Validação	NestJS Docs — Validation
	class-validator (GitHub)
NestJS — Extensibilidade	NestJS Docs — Custom Decorators
	NestJS Docs — Dynamic Modules
	NestJS Docs — Configuration
NestJS — CQRS e Testes	NestJS Docs — CQRS
	NestJS Docs — Testing

Tópico	Links recomendados
Microserviços — Fundamentos	Martin Fowler — Microservices
	Martin Fowler — MonolithFirst
	Sam Newman — Building Microservices
Microserviços — EDA, Saga e NestJS	microservices.io — Saga Pattern
	NestJS Docs — Microservices
Microserviços — Padrões de borda	microservices.io — API Gateway
	Martin Fowler — CircuitBreaker
	opossum
Microserviços — Observabilidade	OpenTelemetry Docs — Tracing

Appendix B

Glossário

Aggregate. O cluster inteiro de objetos de domínio (uma Entity raiz mais suas Entities internas e Value Objects) tratado como uma única unidade de consistência transacional. Ver seção 1.3.

Aggregate Root. A única Entity dentro de um Aggregate que pode ser referenciada e modificada a partir de fora do cluster; todo acesso ao Aggregate passa por ela, que garante as invariantes do grupo inteiro. Ver seção 1.3.

Anticorruption Layer (ACL). Camada de tradução que impede que o vocabulário e as estruturas de dados de um sistema externo (ERP legado, API de terceiro) vazem para dentro do domínio limpo; implementada na prática por um Mapper. Ver seções 1.7 e 2.6.

API Gateway. Ponto único de entrada que roteia requisições externas para o microserviço correto, geralmente cuidando também de autenticação, rate limiting e agregação de respostas. Ver seção 3.6.

Application Service (Use Case). Camada que orquestra a infraestrutura ao redor do domínio: busca agregados no repositório, chama o domínio, salva o resultado, dispara eventos e gerencia transações. Ver seção 1.5.

BFF (Backend for Frontend). Variação do API Gateway com um gateway dedicado por tipo de cliente (mobile, web administrativo, etc.), evitando que um gateway genérico vire um meio-termo ruim para todos. Ver seção 3.6.

Bounded Context. Fronteira dentro da qual um modelo de domínio e sua linguagem são consistentes e válidos; a mesma palavra pode ter significados diferentes em contextos diferentes, e isso é esperado. Ver seções 1.1 e 1.7.

Circuit Breaker. Padrão de resiliência que interrompe temporariamente chamadas a um serviço que está falhando repetidamente, retornando uma falha rápida ou um fallback em vez de acumular timeouts. Ver seção 3.6.

Correlation ID. Identificador único gerado na borda de uma arquitetura distribuída e propagado por todas as chamadas subsequentes entre serviços, permitindo reconstruir a jornada completa de uma requisição em ferramentas de tracing. Ver seção 3.7.

CQRS (Command Query Responsibility Segregation). Padrão que separa ex-

plicitamente operações que mudam estado (Commands) de operações que leem estado (Queries), cada uma com seu próprio caminho de código. Ver seção 2.8.

Domain Event. Objeto imutável, nomeado no passado, que representa algo importante que já aconteceu no domínio; permite que uma Aggregate Root avise o resto do sistema sem conhecer quem está ouvindo. Ver seção 1.6.

Domain Service. Regra de negócio que não pertence naturalmente a nenhuma Entity ou Value Object específico, geralmente porque envolve mais de um agregado; permanece domínio puro, sem dependência de infraestrutura. Ver seção 1.5.

DTO (Data Transfer Object). Objeto simples, sem comportamento de negócio, cuja única responsabilidade é transportar dados através de uma fronteira (tipicamente o corpo de uma requisição HTTP), definindo a forma esperada do dado. Ver seção 2.4.

Entity. Objeto de domínio com identidade própria que persiste ao longo do tempo mesmo que seus atributos mudam; igualdade é definida por identidade, não por atributos. Ver seção 1.2.

Lei de Conway. Observação de que sistemas de software tendem a espelhar a estrutura de comunicação da organização que os constrói; usada como argumento central contra adotar microsserviços sem uma estrutura de times que os justifique. Ver seção 3.1.

Mapper. Lógica que traduz dado bruto e validado (de um DTO ou de um sistema externo) para objetos de domínio (Entities e Value Objects), ou vice-versa; a contraparte comportamental do DTO. Ver seção 2.6.

OpenTelemetry. Padrão aberto de instrumentação para tracing distribuído, sucessor unificado do OpenTracing e OpenCensus, que organiza uma requisição em um trace composto por spans. Ver seção 3.7.

Repository. Abstração que simula uma coleção em memória de Aggregate Roots; sua interface é definida no domínio, e a implementação concreta (ORM, SQL) vive na infraestrutura. Ver seção 1.4.

Saga. Sequência de transações locais entre múltiplos serviços, onde cada etapa publica um evento que dispara a próxima e define uma ação de compensação para desfazer seu efeito caso uma etapa posterior falhe. Ver seção 3.4.

Strangler Fig. Padrão de migração no qual um proxy/gateway é colocado na frente de um monólito e Bounded Contexts são extraídos um de cada vez para serviços novos, até o monólito original encolher a pouco ou nada. Ver seção 3.1.

Ubiquitous Language (Linguagem Ubíqua). Prática de usar exatamente os mesmos termos do negócio no código, para que o código comunique diretamente a regra de negócio em vez de exigir tradução mental. Ver seção 1.1.

Unit of Work. Padrão que rastreia as mudanças feitas durante uma operação e as persiste como uma única transação atômica, garantindo que todos os repositórios envolvidos sejam salvos juntos ou nenhum seja. Ver seção 1.4.

Value Object. Objeto de domínio sem identidade própria, definido inteiramente por seus atributos e imutável; dois Value Objects com os mesmos atributos são considerados

iguais. Ver seção 1.2.

Index

Aggregate, 9
Aggregate Root, 9
Anticorruption Layer (ACL), 21
API Gateway, 58
Application Service, 16

BFF (Backend for Frontend), 59
Bounded Context, 4

Circuit Breaker, 59
Controller (NestJS), 24
Correlation ID, 60
CQRS, 44

Domain Event, 18
Domain Service, 16
DTO, 32

Entity, 6

Lei de Conway, 50

Mapper, 37
Module (NestJS), 24

OpenTelemetry, 61

Provider (NestJS), 24

Repository, 13

Saga, 55
Strangler Fig, 51

Ubiquitous Language, 4
Unit of Work, 15

Value Object, 6